



ENSAIOS EM VOO

Trabalho Experimental

Grupo 24: Gabriel Faria, 109306
Gabriel Leitão, 109835
Rodrigo Boulhosa, 110716

Professor: Agostinho Fonseca

Junho 2026

LEAer
2.º Semestre 2025/2026

Conteúdo

1	Introdução e Objetivos	1
2	Avaliação do desempenho do sistema de navegação EGNOS	2
2.1	Pergunta 1 - Avaliação da exatidão de um sistema de GNSS	2
2.2	Pergunta 2 e 3 - Erros de um sistema GNSS com aproximação Haversine	4
2.3	Pergunta 4 - HPE e VPE do sistema de navegação EGNOS	5
2.4	Pergunta 5 - Análise temporal dos erros de navegação e parâmetros de integridade	5
2.5	Pergunta 6 - Exatidão e integridade do sistema EGNOS	7
2.5.1	Exatidão	7
2.5.2	Integridade	7
2.5.3	Disponibilidade	8
2.5.4	Continuidade	9
2.6	Pergunta 7 - Eventos de integridade	10
2.7	Pergunta 8 - Conclusões	11
3	Sistema de Instrumentação PCM de Classe I	12
3.1	Pergunta 9 - Variação temporal dos sinais	12
3.2	Pergunta 10 - Espectro unilateral de amplitude	13
3.2.1	Transformada de Fourier	13
3.2.2	Espectro unilateral de amplitude	14
3.3	Pergunta 11 - Picos de amplitude	16
3.4	Pergunta 12 - Conclusões	16
4	Análise de dados de voo do Dornier-Dassault Alpha Jet	18
4.1	Pergunta 13 – Variação temporal das grandezas medidas	18
4.2	Pergunta 14 - Conversão da aceleração vertical para unidades g	21
4.3	Pergunta 15 - Extremos relativos da aceleração	22
4.4	Pergunta 16 - Ciclos de aceleração vertical	23
4.5	Pergunta 17 - Pares de valores	23
4.6	Pergunta 18 - Ciclos de aceleração filtrados	24
4.7	Pergunta 19 - Comentários	24
A	Descrição dos campos do ficheiro EV_2026.A24	26
B	Código de Análise do Ficheiro EV_2026.A24	27
C	Código de Análise do Ficheiro EV_2026.B24	35
D	Código de Análise do Ficheiro EV_2026.C24	37

1 Introdução e Objetivos

Este trabalho experimental visa a familiarização com o processamento e análise de dados de instrumentação utilizada no âmbito de Ensaio em Voo. Este relatório exibe a metodologia utilizada no processamento dos dados, bem como os resultados obtidos, de forma clara e instrutiva, de modo a poder ser reproduzida para diferentes *datasets*.

Este documento contém três análises distintas: Avaliação de um sistema de navegação EGNOS; Análise de dados proveniente de um sistema de instrumentação PCM de classe I; e Análise de dados de voo da aeronave DORNIER-DASSAULT ALPHA-JET. (ACMAA 2025a)

Em anexo, encontram-se os programas utilizados no processamento e análise dos dados.

2 Avaliação do desempenho do sistema de navegação EGNOS

Nesta secção pretende-se analisar o desempenho do sistema de navegação EGNOS com base nos dados contidos no ficheiro `EV_2026.A24`, recolhidos durante um voo de calibração de um sistema convencional de navegação. Os dados GNSS foram adquiridos através de um recetor SEPTENTRIO POLARX2, ligado a uma antena NOVATEL L1/L2 GPSANTENNA MODEL 512 REV. 2, com uma frequência de amostragem de 1 Hz.

O ficheiro inclui, para cada instante de tempo, a solução de navegação EGNOS e a respetiva trajetória de referência, considerada como a trajetória verdadeira da aeronave, obtida através de pós-processamento diferencial utilizando o software TOTAL TRIMBLE CONTROL e dados provenientes de estações de referência GNSS. A solução EGNOS foi igualmente gerada em pós-processamento recorrendo ao pacote de software PEGASUS, desenvolvido pelo EUROCONTROL.

Com base nestes dados serão avaliados os erros de posicionamento horizontal (HPE) e vertical (VPE), bem como os respetivos níveis de proteção horizontal (HPL) e vertical (VPL), de forma a verificar a conformidade do sistema com os requisitos estabelecidos pela ICAO para sistemas SBAS. A análise será efetuada considerando os três modos de operação definidos nos *Standards and Recommended Practices* (SARPs) International Civil Aviation Organization 2023:

- *Approach Procedure with Vertical Guidance I* (APV-I)
- *Approach Procedure with Vertical Guidance II* (APV-II)
- *Precision Approach Category I* (CAT-I)

2.1 Pergunta 1 - Avaliação da exatidão de um sistema de GNSS

Antes de iniciar o processamento e análise dos dados disponíveis no ficheiro `EV_2026.A24`, é nos pedido para considerar a antena do recetor de um sistema GNSS colocada numa posição de referência, cujas coordenadas WGS84 são as seguintes:

$$\phi_{\text{ref}} = 38.80572026^\circ \text{ N}$$

$$\lambda_{\text{ref}} = -9.19497112^\circ \text{ W}$$

$$h_{\text{ref}} = 1282.192 \text{ m}$$

e utilizando a metodologia descrita no documento EV – Avaliação da exatidão de um sistema GNSS – 2025-2026.pdf ACMAA 2025b com $\xi = 10$, determinar os erros de posição horizontal e vertical (d e h, respetivamente), para as soluções de navegação A, B, C e D. Também é pedido para apresentar as coordenadas dos vários pontos utilizados nesta análise no referencial cartesiano considerado, em m .

As soluções de navegação fornecidas são apresentadas na Tabela 1.

Tabela 1: Soluções de navegação GNSS

Solução	Latitude (°)	Longitude (°)	Altitude (m)
A	38.80482050	-9.19496103	1284.221
B	38.80572550	-9.19497203	1281.321
C	38.80581050	-9.19497603	1280.221
D	38.80562050	-9.19497003	1282.221

Começou-se por estudar a transformação de coordenadas que permitia passar de um referencial elipsoidal (WGS84) para um referencial cartesiano (ECI). Essa transformação é dada por:

$$X = (v + h) \cos \phi \cos \lambda$$

$$Y = (v + h) \cos \phi \sin \lambda$$

$$Z = [v(1 - e^2) + h] \sin \phi$$

com

$$v = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}}, \quad a = 6\,378\,137 \text{ m}, \quad e^2 = 6.69437999014 \times 10^{-3}.$$

De modo a validar esta transformação definiu-se a função `wgs84_latlonh_to_xyz(lon, lat, h)` e utilizou-se **Transformer** da biblioteca *Python Pyproj*. Considerando um ponto qualquer sobre o elipsoide e fazendo a conversão utilizando os dois métodos, apenas existem diferenças na ordem de centímetros. Para um rigor superior, será então utilizada a função **Transformer**. (Stack Overflow Contributors 2010, EPSG.io 2026a, EPSG.io 2026b, Quora Contributors 2020)

Definiu-se também a função `get_error(xi, lon_ant, lat_ant, hgt_ant, lon, lat, hgt)` que recebe o parâmetro ξ , a localização da antena e das soluções do sistema GNSS e retorna o erro horizontal e vertical. Esta função utiliza a biblioteca *Python Numpy* para facilitar a sua implementação.

Obtiveram-se os seguintes resultados:

Tabela 2: Coordenadas ECEF e erros das soluções de navegação

Ponto	X (m)	Y (m)	Z (m)	d_{error} (m)	h_{error} (m)
Antena	4913906.890	-795436.646	3976336.319	-	-
Solução A	4913970.394	-795446.037	3976259.738	99.901	2.356
Solução B	4913905.847	-795436.557	3976336.227	0.584	-0.873
Solução C	4913899.107	-795435.818	3976342.892	10.022	-2.004
Solução D	4913913.780	-795437.665	3976327.706	11.077	0.065

2.2 Pergunta 2 e 3 - Erros de um sistema GNSS com aproximação Haversine

Na pergunta 4, será utilizada a equação de Haversine para calcular o erro horizontal das soluções obtidas pelo sistema GNSS. É-nos pedido previamente que se utilize esta ferramenta para determinar os erros das soluções de navegação anteriormente referidas. Para tal definiu-se a função `haversine(ref_lat, ref_lon, ns_lat, ns_lon)`, que recebe a latitude e a longitude de dois pontos, e retorna o erro horizontal (HPE), que é dado pelas seguintes expressões:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right), \quad \Delta\varphi = \varphi_2 - \varphi_1, \quad \Delta\lambda = \lambda_2 - \lambda_1,$$

$$c = 2 \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right)$$

$$HPE = R c$$

sendo φ_1 e λ_1 a latitude e longitude da posição de referência, φ_2 e λ_2 a latitude e longitude da solução EGNOS, e $R = 6\,371\,000$ m o raio médio da Terra.

O erro vertical é simplesmente dado por:

$$VPE = NS_ALT - REF_ALT$$

Tabela 3: Comparação entre os erros originais e os erros calculados pela fórmula de Haversine

Solução	d_{error} (m)	h_{error} (m)	$d_{\text{error,Hav}}$ (m)	$h_{\text{error,Hav}}$ (m)
A	99.9008	2.3556	100.0526	2.0290
B	0.5843	-0.8729	0.5880	-0.8710
C	10.0223	-2.0038	10.0432	-1.9710
D	11.0770	0.0653	11.0932	0.0290

Observa-se que os valores obtidos através da fórmula de Haversine são muito próximos dos erros horizontais originais, apresentando diferenças inferiores a alguns centímetros para as soluções B, C e D, e da ordem dos decímetros para a solução A. Estes resultados confirmam que o método proposto para o cálculo dos erros horizontais e verticais constitui uma boa aproximação para posições geográficas próximas.

Verifica-se ainda que, à medida que o erro horizontal aumenta, as diferenças observadas no erro vertical tendem também a aumentar ligeiramente. Este comportamento é expectável, uma vez que a fórmula de Haversine calcula a distância sobre a superfície terrestre assumindo um modelo esférico da Terra, enquanto a componente vertical é determinada relativamente ao elipsoide de referência WGS84. Consequentemente, para separações horizontais maiores, surgem pequenas discrepâncias geométricas associadas à não ortogonalidade entre as direções locais e as aproximações utilizadas. No entanto, estas diferenças mantêm-se reduzidas, da ordem de alguns centímetros, sendo desprezáveis quando comparadas com as dimensões características de uma aeronave e com os níveis de precisão normalmente exigidos em aplicações de navegação aérea.

2.3 Pergunta 4 - HPE e VPE do sistema de navegação EGNOS

Para a determinação dos erros de navegação foi utilizado o método de Haversine, previamente descrito, para o cálculo do erro horizontal. O ficheiro fornecido contém informação temporal, parâmetros de integridade (HPL e VPL), soluções de navegação EGNOS e a respetiva trajetória de referência. A descrição completa dos campos encontra-se no Anexo A.

Para a leitura e processamento do ficheiro `EV_2026.A24` foi utilizada a biblioteca *Python Pandas*, tendo os dados sido armazenados num *DataFrame*. Posteriormente, recorreu-se à biblioteca *Numpy* para a extração das variáveis do *DataFrame* para estruturas do tipo lista, de forma a facilitar o processamento numérico.

O erro horizontal (HPE) foi calculado através da fórmula de Haversine: `haversine(REF_LAT, REF_LON, NS_LAT, NS_LON)`. O erro vertical (VPE) foi obtido pela diferença entre a altitude da solução EGNOS, `NS_ALT`, e a altitude da trajetória de referência, `REF_ALT`.

2.4 Pergunta 5 - Análise temporal dos erros de navegação e parâmetros de integridade

Para a representação gráfica das variáveis em função do tempo, a variável `RX_TOM` foi convertida para um formato temporal adequado à análise (`YYYY-MM-DD HH:MM:SS`).

Considerando que o sistema WGS84 utiliza como época de referência o instante `Epoch = 1980-01-06`, e que os dados do ensaio estão associados à semana GPS `RX_WEEK`, foi possível converter `RX_TOM` em *timestamps* absolutos.

Posteriormente, recorreu-se à função `time_format()` para proceder à formatação adequada das variáveis temporais utilizadas na construção dos gráficos, permitindo uma melhor interpretação da evolução temporal dos erros, dos níveis de proteção e do número de satélites utilizados na solução de navegação EGNOS.

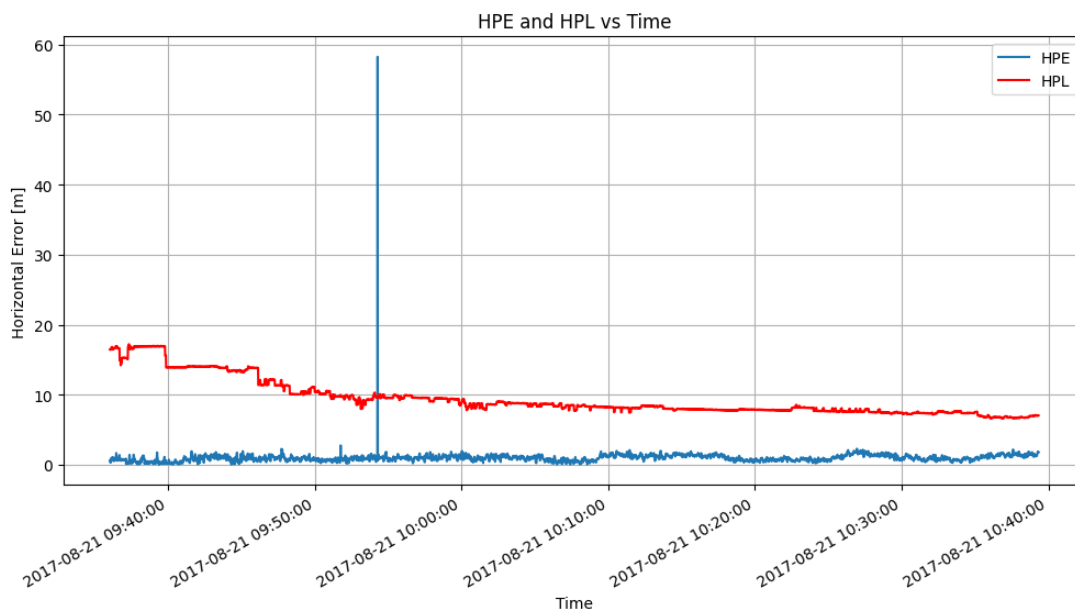


Figura 1: Evolução temporal de HPE e HPL ao longo do tempo.

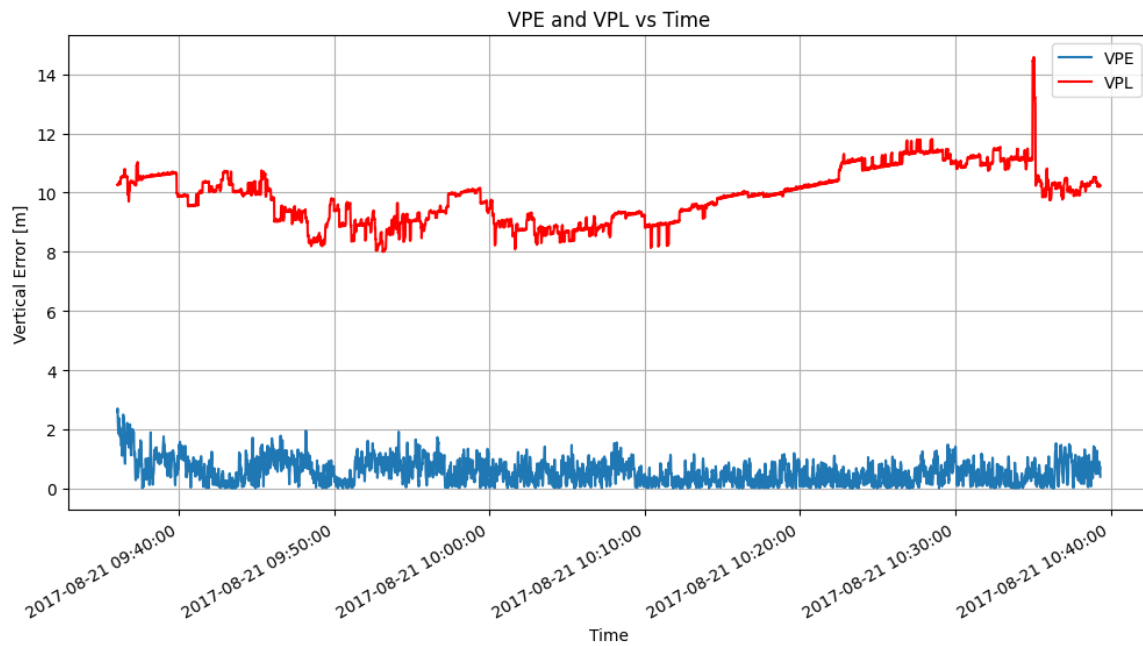


Figura 2: Evolução temporal de VPE e VPL ao longo do tempo.

Note-se que usa-se o módulo de VPE na representação gráfica para ser possível comparar com o correspondente nível de proteção. Para a representação do número de satélites utilizados na determinação da solução EGNOS foi utilizada a variável `NSV_USED`, enquanto que o número de satélites observados em cada instante de tempo é dado por `NSV_LOCK`.

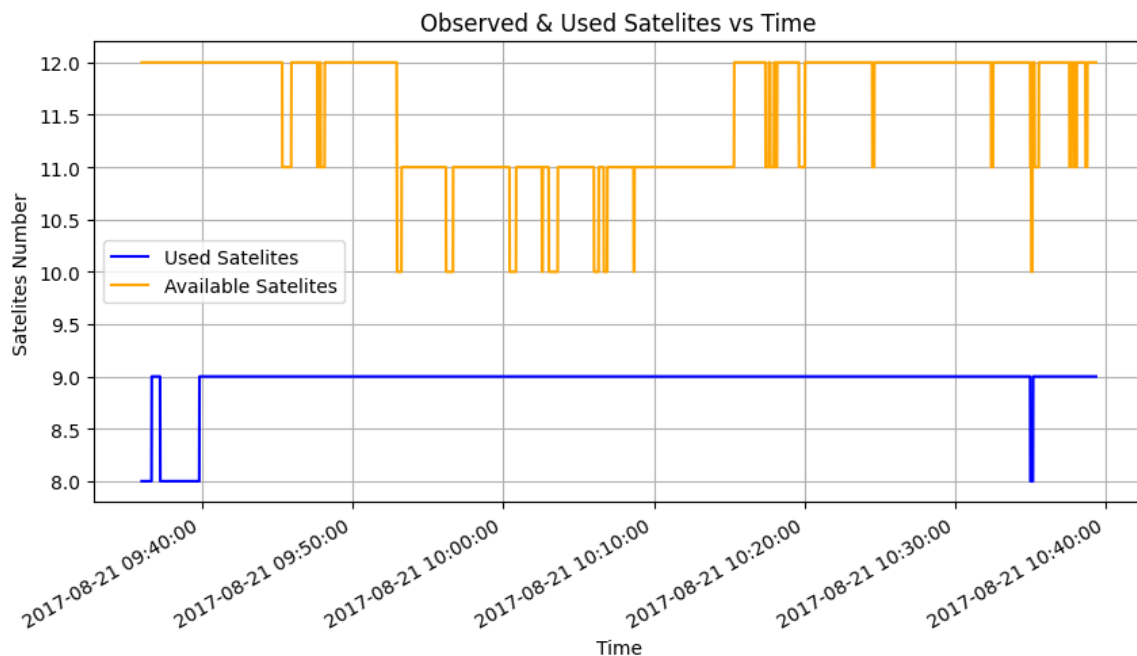


Figura 3: Evolução temporal do número de satélites observados (`NSV_LOCK`) e utilizados (`NSV_USED`) na solução de navegação EGNOS ao longo do tempo.

2.5 Pergunta 6 - Exatidão e integridade do sistema EGNOS

Nesta secção procede-se à avaliação dos parâmetros de desempenho do sistema de navegação EGNOS, nomeadamente a exatidão e a integridade, com base nos requisitos definidos pela ICAO para sistemas SBAS. A análise é efetuada através dos erros de posicionamento horizontal (Horizontal Position Error – HPE) e vertical (Vertical Position Error – VPE), bem como dos níveis de proteção horizontal e vertical (HPL e VPL).

2.5.1 Exatidão

A exatidão do sistema é avaliada através dos erros de posição horizontal e vertical, sendo posteriormente calculados os percentis de 95% das amostras. Estes valores são comparados com os limites definidos pela ICAO para os diferentes modos de operação SBAS.

Tabela 4: Limites de exatidão para sistemas SBAS segundo a ICAO

Modo de operação	HPE (95%)	VPE (95%)
APV-I	16 m	20 m
APV-II	16 m	8 m
CAT-I	16 m	5 m

Os valores obtidos para os percentis de 95% foram:

$$HPE_{95} = 1.63 \text{ m}, \quad VPE_{95} = 1.26 \text{ m}$$

Verifica-se que ambos os valores se encontram significativamente abaixo dos limites estabelecidos para todos os modos de operação, pelo que o sistema EGNOS cumpre os requisitos de exatidão definidos pela ICAO.

2.5.2 Integridade

A integridade do sistema é avaliada através dos níveis de proteção horizontal (HPL) e vertical (VPL), sendo analisados os percentis de 99% e comparados com os respetivos níveis de alerta.

Tabela 5: Limites de integridade para sistemas SBAS segundo a ICAO

Modo de operação	HAL	VAL
APV-I	40 m	50 m
APV-II	40 m	20 m
CAT-I	40 m	12 m

Os valores obtidos foram:

$$HPL_{99} = 16.92 \text{ m}, \quad VPL_{99} = 11.69 \text{ m}$$

Verifica-se que o percentil de 99% do HPL se encontra significativamente abaixo do limite de 40 m definido para todos os modos de operação. No caso do VPL, observa-se igualmente conformidade com o requisito mais exigente (CAT-I), cujo limite é de 12 m.

Deste modo, conclui-se que o sistema EGNOS cumpre os requisitos de integridade definidos pela ICAO, apresentando margens de segurança adequadas mesmo para operações de aproximação de precisão.

2.5.3 Disponibilidade

A disponibilidade é definida como a percentagem de tempo, relativamente ao período de análise, em que o sistema de navegação permite a utilização de um determinado modo de operação. Considera-se que, num dado instante, o sistema suporta o modo de operação caso os níveis de proteção horizontal e vertical (HPL e VPL) sejam inferiores aos respectivos níveis de alerta (HAL e VAL).

De acordo com os requisitos definidos pela ICAO para sistemas SBAS, um modo de operação é considerado operacionalmente viável caso a disponibilidade seja superior a 99%.

Para a análise da disponibilidade foram considerados os níveis de alerta definidos anteriormente para os diferentes modos de operação:

Tabela 6: Níveis de alerta para avaliação da disponibilidade

Modo de operação	HAL	VAL
APV-I	40 m	50 m
APV-II	40 m	20 m
CAT-I	40 m	12 m

A verificação da condição de disponibilidade foi realizada através da comparação temporal entre os níveis de proteção e os respectivos níveis de alerta, como ilustrado nas Figuras 4 e 5.

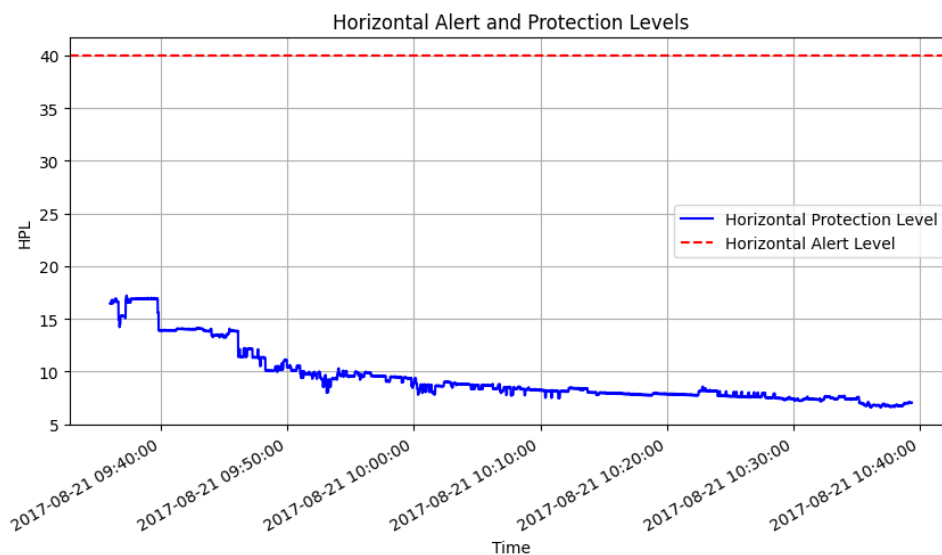


Figura 4: Comparação entre HPL e HAL.

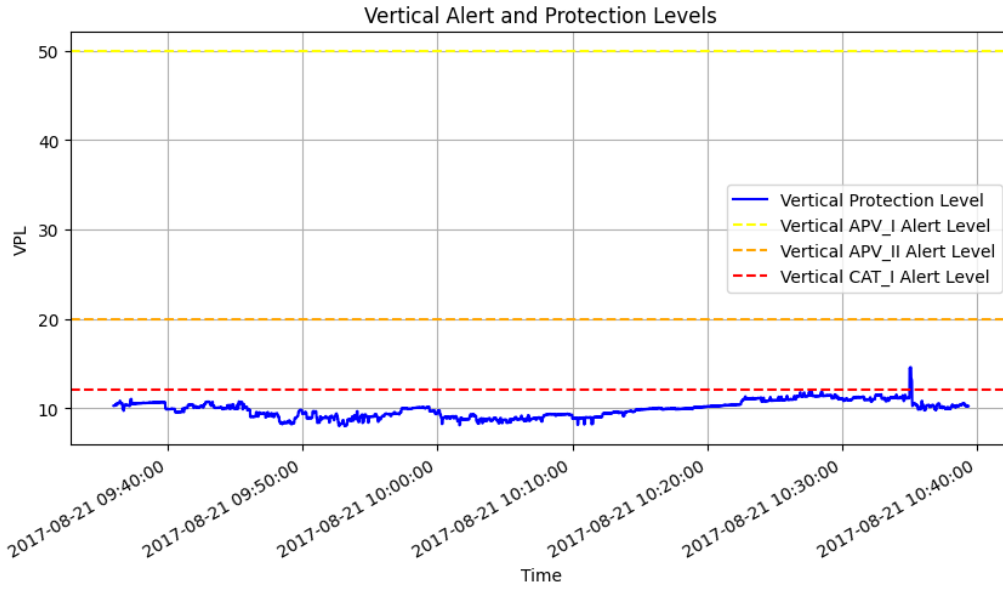


Figura 5: Comparação entre VPL e VAL.

A disponibilidade foi então calculada através da função:

$$\text{Disponibilidade} = \frac{N_{\text{válidos}}}{N_{\text{total}}} \times 100$$

onde um instante é considerado válido quando $HPL < HAL$ e $VPL < VAL$.

Os resultados obtidos foram:

$$\text{APV-I} = 100.0\%, \quad \text{APV-II} = 100.0\%, \quad \text{CAT-I} = 99.71\%$$

Observa-se que os níveis de proteção se mantêm, na generalidade do período de análise, abaixo dos respetivos níveis de alerta. Assim, os modos APV-I e APV-II apresentam disponibilidade total (100%), enquanto o modo CAT-I apresenta uma ligeira degradação pontual associada à ultrapassagem do limiar vertical.

Contudo, mesmo para o modo CAT-I, a disponibilidade obtida (99.71%) encontra-se acima do limiar mínimo de 99% definido pela ICAO, pelo que o sistema EGNOS cumpre os requisitos de disponibilidade para todos os modos de operação analisados. A disponibilidade foi calculada recorrendo à função `get_availability(RX_TOMf, HALf, VALf, HPLf, VPLf)`.

2.5.4 Continuidade

A continuidade do serviço é avaliada através da contabilização dos eventos de continuidade, correspondentes aos intervalos temporais durante os quais o sistema se manteve disponível para um determinado modo de operação. Para tal, foi desenvolvida a função `get_continuity()`, que identifica os períodos de disponibilidade com base nos mesmos critérios utilizados na análise da disponibilidade, isto é, verificando se os níveis de proteção horizontal e vertical permanecem abaixo dos respetivos níveis de alerta.

Os resultados obtidos para cada modo de operação foram:

$$\text{APV-I} = 1, \quad \text{APV-II} = 1, \quad \text{CAT-I} = 2$$

Observa-se que os modos de operação APV-I e APV-II permaneceram disponíveis durante todo o período de ensaio, apresentando apenas um único intervalo de continuidade. Este resultado é consistente com a disponibilidade de 100% anteriormente verificada para ambos os modos de operação.

Por outro lado, o modo CAT-I apresenta dois intervalos de continuidade, evidenciando a ocorrência de uma interrupção temporária da disponibilidade durante o ensaio. Esta interrupção pode ser observada na Figura 5, onde o nível de proteção vertical excede pontualmente o correspondente nível de alerta. Verifica-se ainda que esta degradação está associada exclusivamente à componente vertical da solução, uma vez que o requisito horizontal foi satisfeito durante todo o período de observação.

2.6 Pergunta 7 - Eventos de integridade

A identificação de eventos de integridade foi realizada através da comparação entre os erros de navegação e os correspondentes níveis de proteção. De acordo com a definição apresentada, considera-se que ocorre um evento de integridade quando, num determinado instante, o erro horizontal excede o nível de proteção horizontal ($HPE > HPL$) ou quando o erro vertical excede o nível de proteção vertical ($VPE > VPL$).

Para efetuar esta análise foi desenvolvida a função `check_integrity()`, que percorre todas as amostras do conjunto de dados, identifica eventuais violações dos critérios de integridade e regista o instante temporal correspondente. Adicionalmente, a função classifica cada evento de acordo com a sua origem: violação horizontal, violação vertical ou violação simultânea de ambas as componentes.

Da aplicação desta metodologia aos dados fornecidos resultou a identificação de um único evento de integridade. As características relevantes desse evento encontram-se resumidas na Tabela 7.

Tabela 7: Evento de integridade identificado durante o ensaio

Data e hora	Tipo de violação	HPE (m)	VPE (m)
2017-08-21 09:54:16	Violação HPL	58.23	0.95

Observa-se que o evento corresponde exclusivamente a uma violação da componente horizontal, verificando-se que o erro horizontal ultrapassou o respetivo nível de proteção. Por outro lado, o erro vertical manteve-se dentro dos limites previstos pelo sistema de integridade.

Foi identificado apenas um evento de integridade em todo o período de ensaio, o que evidencia um desempenho globalmente robusto do sistema EGNOS. O erro horizontal observado, de aproximadamente 58 m, corresponde a uma ocorrência pontual que poderá estar associada a fatores como degradação momentânea da geometria dos satélites, efeitos de propagação do sinal, interferências eletromagnéticas ou limitações temporárias dos algoritmos de processamento da solução de navegação. Apesar desta ocorrência isolada, os resultados obtidos indicam que o sistema manteve um comportamento consistente com os requisitos de desempenho analisados nas secções anteriores.

2.7 Pergunta 8 - Conclusões

Esta primeira parte do trabalho experimental permitiu compreender na prática como se avalia o desempenho de um sistema SBAS (EGNOS).

Através do processamento de dados provenientes de um ensaio em voo, foi possível implementar e validar metodologias analíticas de cálculo de erros de posicionamento, nomeadamente a conversão entre referenciais geodésicos e cartesianos, bem como a utilização da aproximação de Haversine. Os exercícios demonstraram a importância de confrontar continuamente os erros de navegação (HPE e VPE) com os respetivos níveis de proteção (HPL e VPL).

A extração e análise das métricas de desempenho exigidas pela ICAO, como a exatidão, integridade, disponibilidade e continuidade, evidenciou a complexidade e o rigor necessários para garantir que um sistema é de facto seguro, para suportar manobras de aproximação de diferentes categorias (APV-I, APV-II e CAT-I).

Em suma, a metodologia desenvolvida e a automação criada através do código *Python* permitem uma análise crítica da robustez e a viabilidade operacional de qualquer sistema GNSS, constituindo uma ferramenta de análise essencial no âmbito de Ensaio em Voo.

O código desenvolvido pode ser consultado em Anexo B.

3 Sistema de Instrumentação PCM de Classe I

Esta secção pretende analisar o ficheiro EV_2026.B24 que contém dados de sistema de instrumentação PCM de Classe I. O ficheiro inclui 5 parâmetros: o tempo, t , em segundos e 4 parâmetros correspondentes a sinais de aceleração, a_1 , a_2 , a_3 e a_4 , expressos em m/s^2 . Os dados foram amostrados em cada instante com período 0.001s, ao longo de 10s. A frequência de amostragem é, portanto, 1000 Hz.

Foi desenvolvido um *script* em *Python* para: 1) ler os dados do ficheiro csv, com separador ponto-e-vírgula; 2) representar graficamente a variação temporal das grandezas; 3) determinar e apresentar o espectro unilateral de amplitude do sinal; 3) e para identificar os picos do espectro na amplitude e as suas frequências.

3.1 Pergunta 9 - Variação temporal dos sinais

A variação temporal dos sinais a_1 a a_4 está representada nas Figuras 6 a 9.

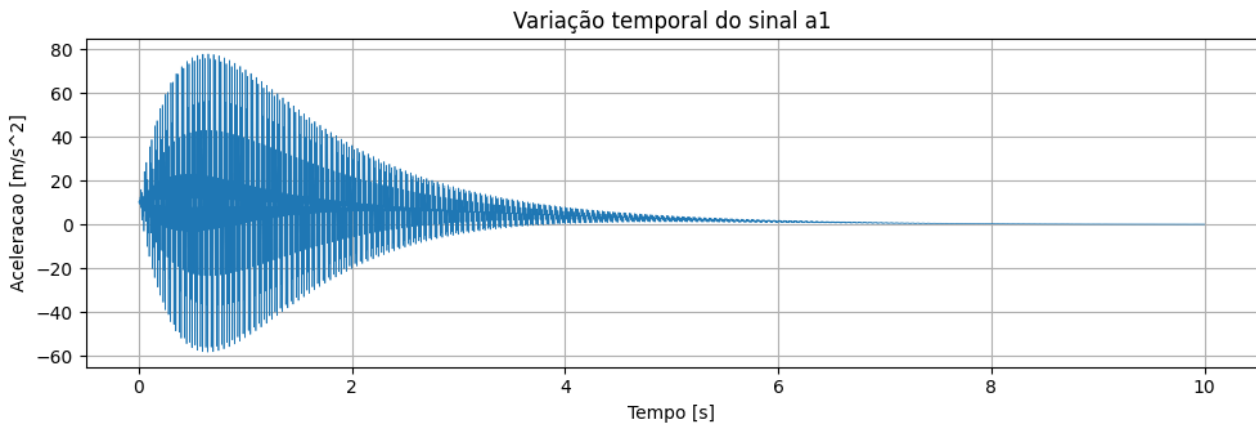


Figura 6: Variação temporal do sinal a_1

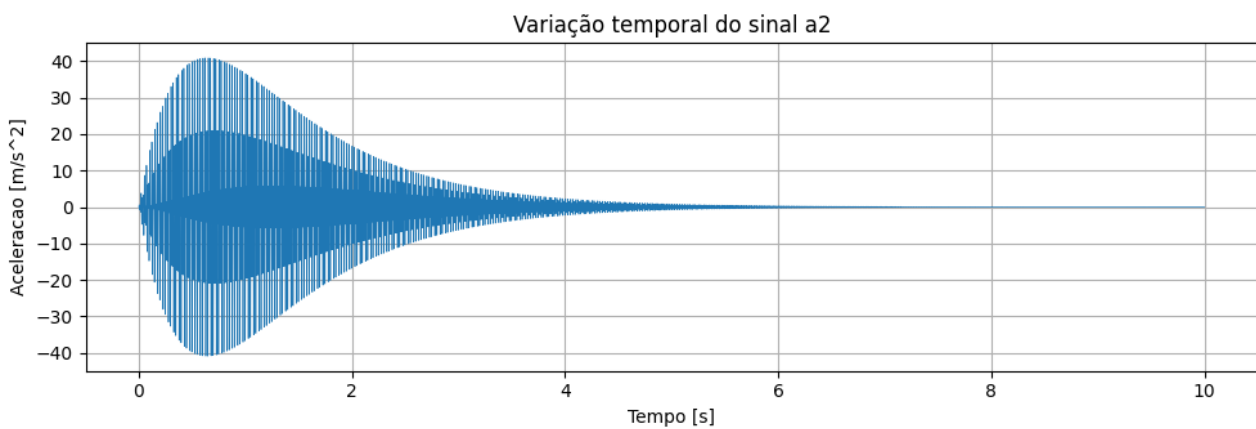
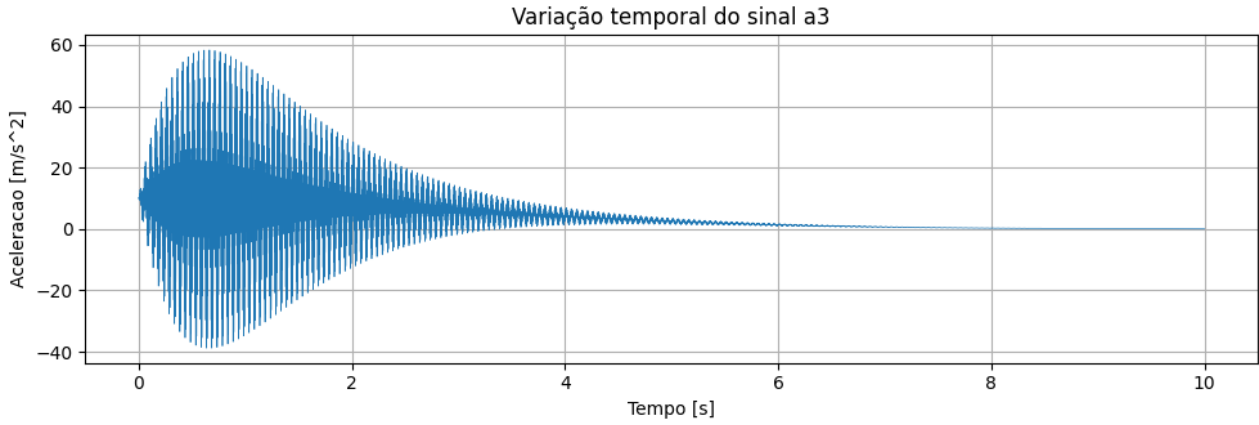
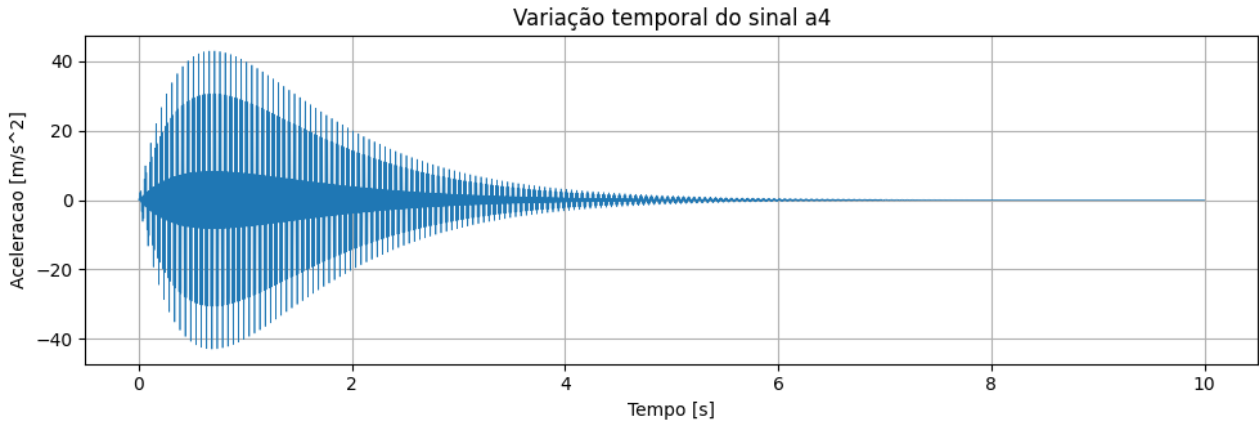


Figura 7: Variação temporal do sinal a_2

Figura 8: Variação temporal do sinal a_3 Figura 9: Variação temporal do sinal a_4

Analisando os gráficos, verifica-se que os vários sinais apresentam um crescimento significativo da sua amplitude máxima até 0,75 s e um decaimento progressivo até 5 s aproximadamente.

Os sinais têm constantemente um comportamento oscilatório, sendo que os sinais a_1 e a_3 começam a oscilar a partir de 10 m/s^2 , enquanto os sinais a_2 e a_4 começam a partir de aceleração nula. No entanto, todos os sinais acabam por convergir para aceleração nula.

3.2 Pergunta 10 - Espectro unilateral de amplitude

3.2.1 Transformada de Fourier

A Transformada de Fourier em frequência angular de uma função é dada por:

$$\mathcal{F}_1\{x(t)\}(\omega) = \int_{-\infty}^{+\infty} x(t)e^{-j\omega t} dt \quad (1)$$

Por exemplo, para $x(t) = \cos(\omega_0 t) = \frac{e^{j\omega_0 t} + e^{-j\omega_0 t}}{2}$:

$$\mathcal{F}_1\{x(t)\}(\omega) = \frac{1}{2} \int_{-\infty}^{+\infty} (e^{j\omega_0 t} + e^{-j\omega_0 t}) e^{-j\omega t} dt \quad (2)$$

$$= \frac{1}{2} \int_{-\infty}^{+\infty} e^{-j(\omega - \omega_0)t} dt + \frac{1}{2} \int_{-\infty}^{+\infty} e^{-j(\omega + \omega_0)t} dt \quad (3)$$

$$= \pi\delta(\omega - \omega_0) + \pi\delta(\omega + \omega_0) \quad (4)$$

onde $\delta(x)$ é a função delta de Dirac. Considerando $\omega_0 = 2\pi f_0$, a Transformada de Fourier em frequência usual é:

$$\mathcal{F}_2\{x(t)\}(f) = \int_{-\infty}^{+\infty} x(t) e^{-j2\pi f t} dt = \mathcal{F}_1\{2\pi f\} \quad (5)$$

Então, a última equação 4 dará, sabendo que $\delta(\alpha x) = \delta(x)/|\alpha|$:

$$\mathcal{F}_2\{x(t)\}(f) = \pi\delta(2\pi f - 2\pi f_0) + \pi\delta(2\pi f + 2\pi f_0) \quad (6)$$

$$= \frac{1}{2}\delta(f - f_0) + \frac{1}{2}\delta(f + f_0) \quad (7)$$

Assim, considerando o gráfico da amplitude do resultado da Transformada de Fourier, de um sinal periódico sinusoidal simples de frequência 50 Hz, e.g., iremos observar dois picos em 50 Hz e -50 Hz. E para qualquer sinal real a Transformada de Fourier produz um gráfico simétrico nas frequências negativas em relação às frequências positivas, pelo que nos interessa considerar apenas o espectro unilateral de amplitude.

3.2.2 Espectro unilateral de amplitude

Para determinar o espectro unilateral foi escolhido o lado das frequências positivas, assim desde a frequência DC 0 até à frequência de Nyquist. No entanto, se apenas cortássemos esses dados do gráfico, a amplitude dos Picos de amplitude no espectro corresponderia a metade da amplitude do sinal, segundo a equação 7. Assim, todas as amplitudes que estariam repetidas nas frequências negativas e positivas foram multiplicadas por 2. Ficam de fora: 1) a frequência zero, DC, pois é única, e 2) a frequência de Nyquist que é única apenas se o número de pontos do sinal, N , for par, segundo a documentação da função `fft()`, que é uma implementação do algoritmo Fast Fourier Transform (FFT) da biblioteca *Python SciPy*. O FFT é o algoritmo mais eficiente para calcular a Transformada de Fourier Discreta, ou a sua inversa, com complexidade computacional $\mathcal{O}(n \log n)$, onde n é o comprimento da sequência de dados.

Assim, o gráfico final do espectro unilateral de amplitude obtido é:

$$G(f) = \begin{cases} 2|\mathcal{F}_2(f)| & \text{se } f \neq 0 \text{ ou } (f = f_s/2 \text{ e } N \text{ é ímpar}) \\ |\mathcal{F}_2(f)| & \text{caso contrário} \end{cases} \quad (8)$$

Neste caso, os sinais têm 10001 pontos e o período de amostragem é 0.001 s, i.e., a frequência de amostragem é 1000 Hz, pelo que a frequência de Nyquist de 500 Hz iria existir também nas frequências negativas, e também por isso foi multiplicada por 2.

Finalmente, o espectro unilateral de amplitude dos sinais estão representados nas Figuras 10 à 13, onde já foi identificado, com cruzeiros laranjas, os picos de amplitude, utilizando a função `find_peaks()` da mesma biblioteca.

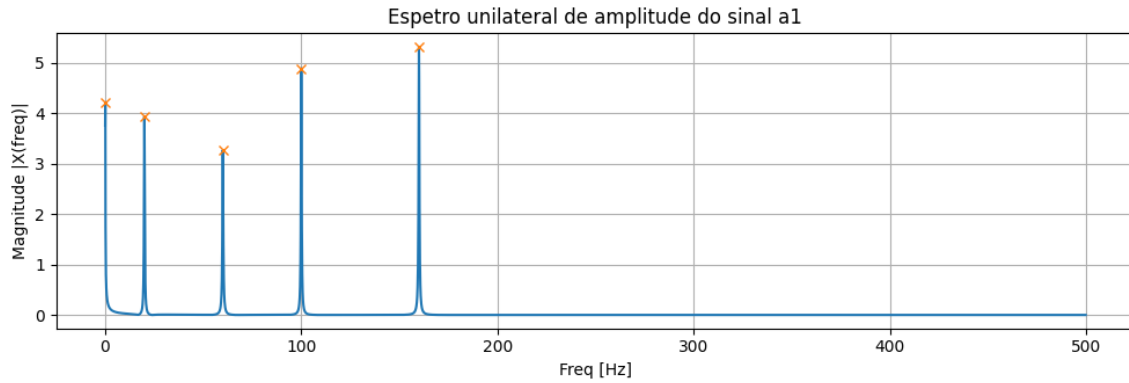


Figura 10: Espectro unilateral de amplitude do sinal a_1

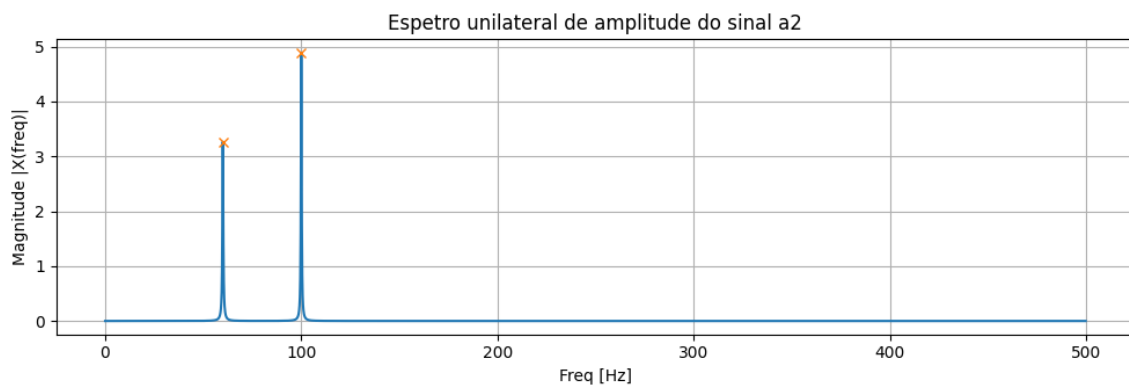


Figura 11: Espectro unilateral de amplitude do sinal a_2

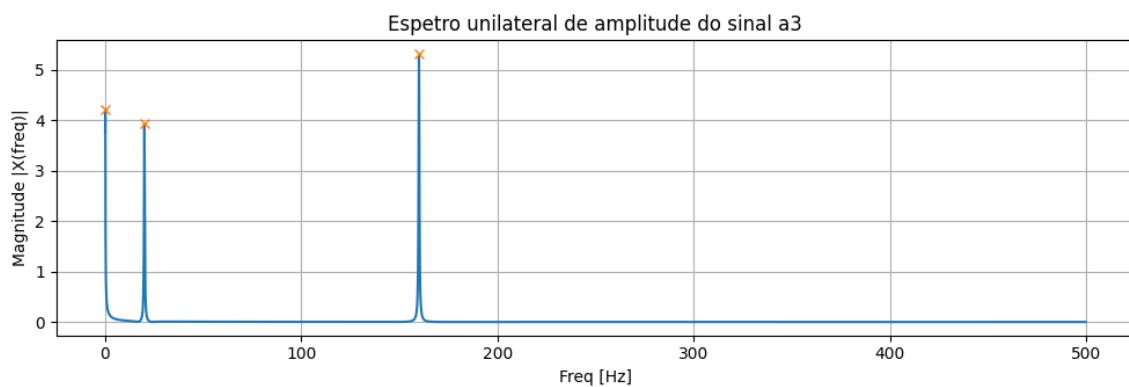


Figura 12: Espectro unilateral de amplitude do sinal a_3

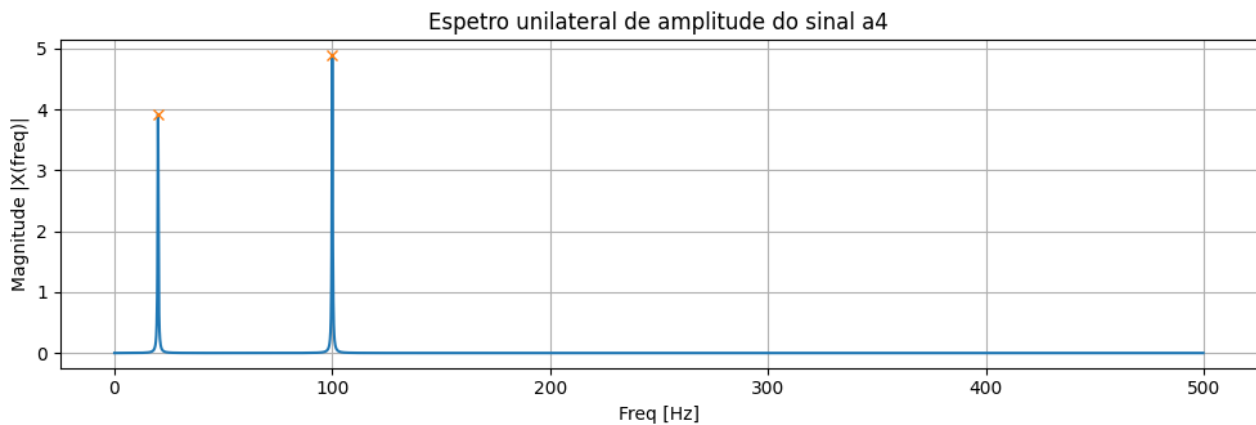


Figura 13: Espectro unilateral de amplitude do sinal a_4

3.3 Pergunta 11 - Picos de amplitude

Como referido acima, os picos de amplitude no espectro foram identificados nos gráficos das Figuras 10 à 13, e são apresentados nas Tabelas 8 à 11:

Frequência (Hz)	Amplitude (m s^{-2})
0.1000	4.2213
19.9980	3.9374
59.9940	3.2715
99.9900	4.8933
159.9840	5.3225

Tabela 8: Picos de amplitude no espectro do sinal a_1

Frequência (Hz)	Amplitude (m s^{-2})
59.9940	3.2664
99.9900	4.8903

Tabela 9: Picos de amplitude no espectro do sinal a_2

Frequência (Hz)	Amplitude (m s^{-2})
0.1000	4.2213
19.9980	3.9375
159.9840	5.3226

Tabela 10: Picos de amplitude no espectro do sinal a_3

Frequência (Hz)	Amplitude (m s^{-2})
19.9980	3.9216
99.9900	4.8904

Tabela 11: Picos de amplitude no espectro do sinal a_4

3.4 Pergunta 12 - Conclusões

Da pergunta 10 observamos espectros limpos, ou seja, a ausência de pequenos picos de amplitude no espectro mostram um sinal com pouco ruído.

Assim, com a informação da pergunta 11, poderemos concluir que os sinais a_1 a a_4 são gerados através da combinação de sinais com 5, 2, 3, 2 frequências diferentes, respetivamente.

Cada um destes 4 sinais compostos pode ser gerado pela sobreposição de ondas sinusoidais com fatores de crescimento e decaimento, aplicados ao sinal composto. Além disso, o offset DC dos sinais a_1 e a_3 também terá um termo de decaimento.

O sinal a_1 contém as mesmas frequências que os restantes sinais, pelo que poderá ser a sobreposição dos sinais a_2 e a_3 , por exemplo.

O código desenvolvido pode ser consultado em Anexo C.

4 Análise de dados de voo do Dornier-Dassault Alpha Jet

Nesta secção, analisa-se o ficheiro EV_2026.C24, que contém dados obtidos em voo numa aeronave Dornier-Dassault Alpha Jet. O ficheiro inclui dez parâmetros: o tempo (t), em s ; a velocidade equivalente do ar (EAS), em kn ; a altitude barométrica (QNE), em ft ; a aceleração vertical (a_z), em m/s^2 ; a velocidade de rotação do motor direito ($N2_{rh}$), em $\%$; a temperatura dos gases de escape do motor direito (EGT_{rh}), em K ; o fluxo de combustível do motor direito (FF_{rh}), em kg/h ; a velocidade de rotação do motor esquerdo ($N2_{lt}$), em $\%$; a temperatura dos gases de escape do motor esquerdo (EGT_{lt}), em K ; e o fluxo de combustível do motor esquerdo (FF_{lt}), em kg/h .

4.1 Pergunta 13 – Variação temporal das grandezas medidas

Os dados obtidos foram para um ensaio em voo que durou cerca de 4118 segundos, com uma frequência de amostragem de aproximadamente 0.5Hz, obtendo-se assim um total de 2257 amostras. A variação temporal dos sinais está representada nas Figuras 14 a 22.

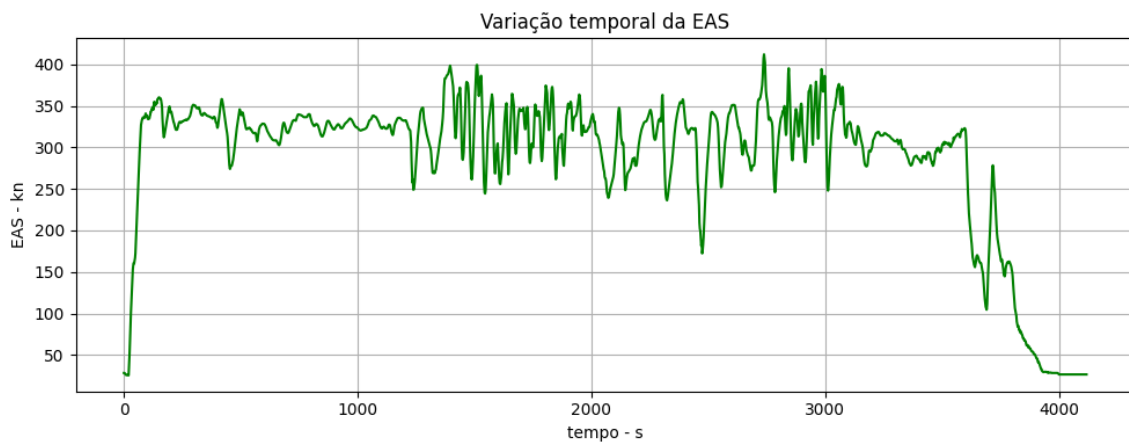


Figura 14: Velocidade EAS

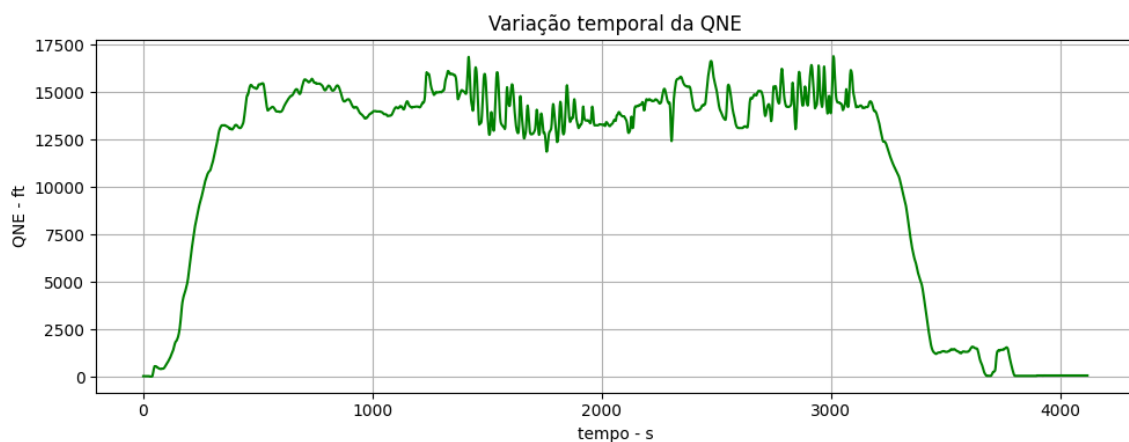
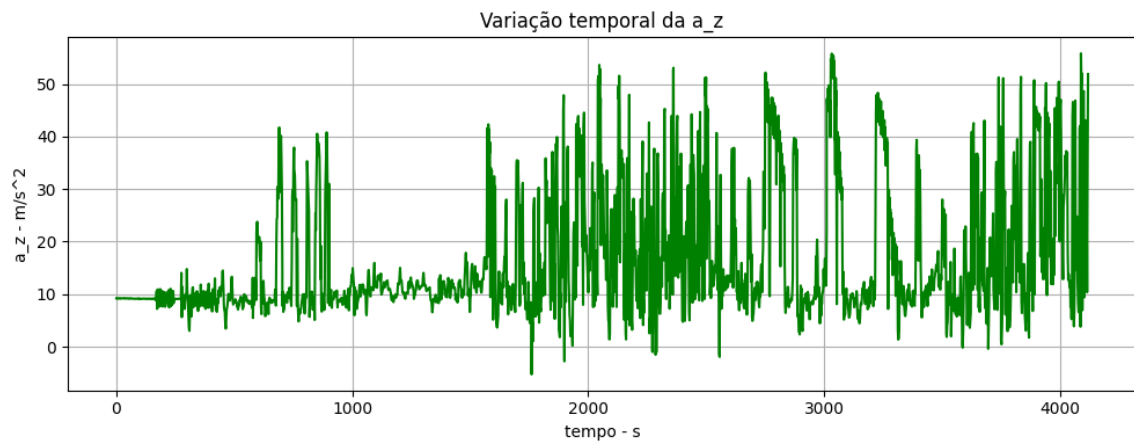
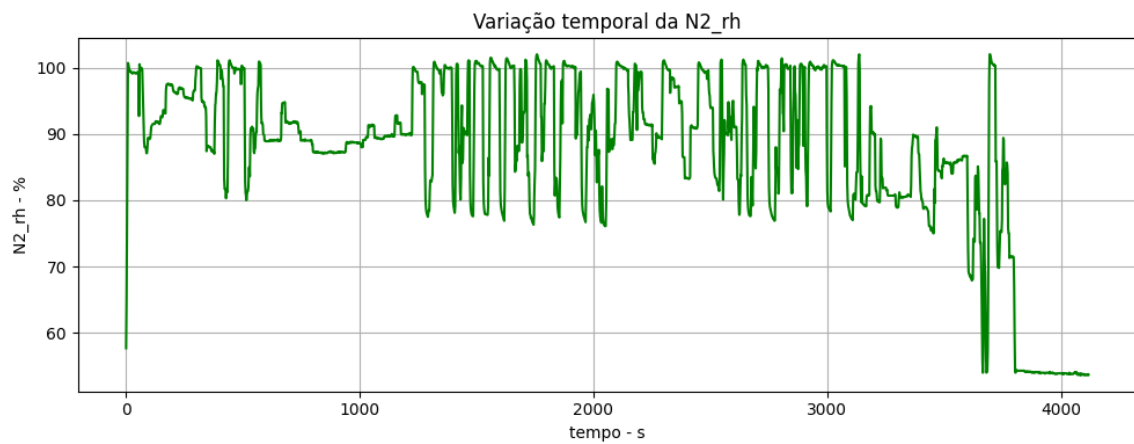
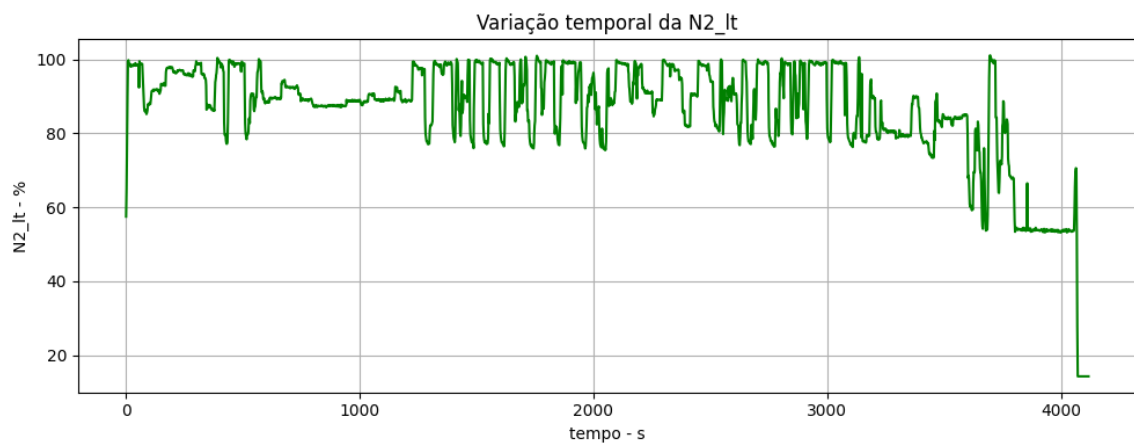
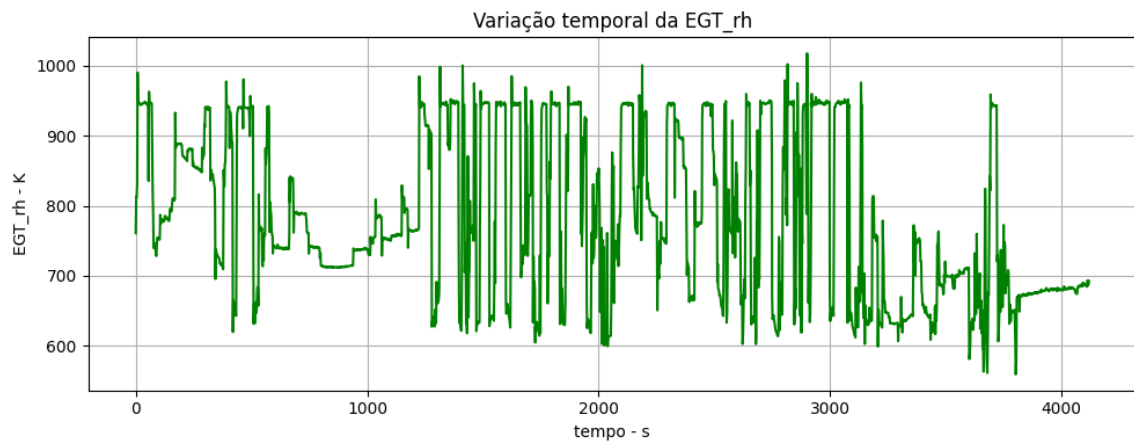
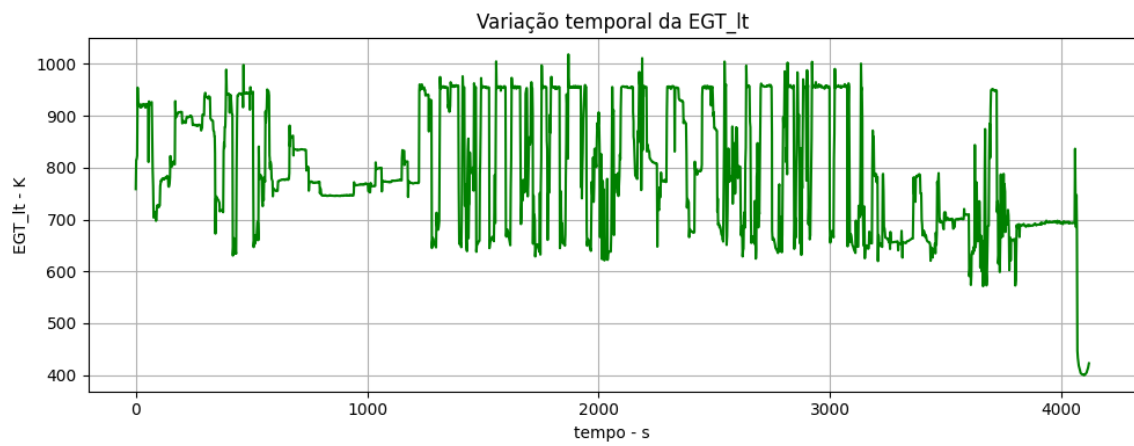
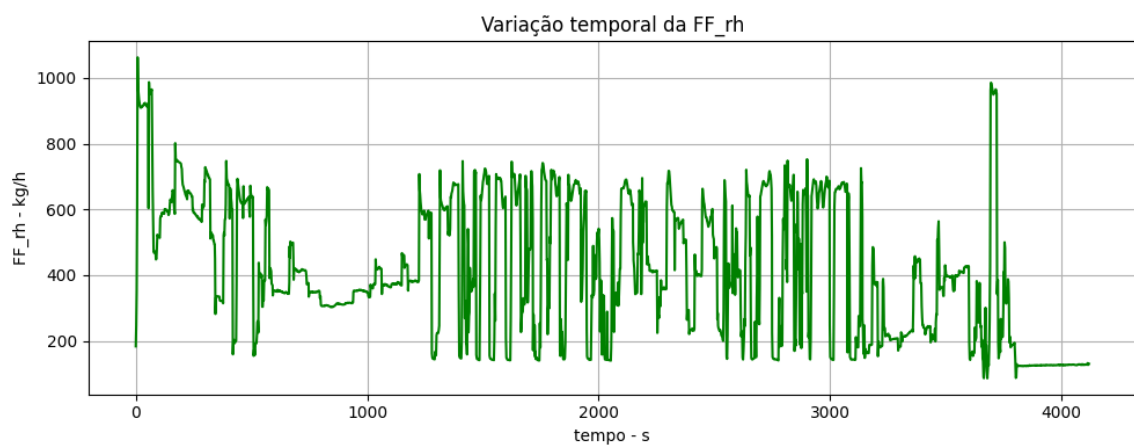


Figura 15: Altitude QNE

Figura 16: Variação temporal da a_z Figura 17: Velocidade de rotação do $N2_{rh}$ Figura 18: Velocidade de rotação do $N2_{lt}$

Figura 19: Temperatura dos gases de escape EGT_{rh} Figura 20: Temperatura dos gases de escape EGT_{lt} Figura 21: Consumo de combustível FF_{rh}

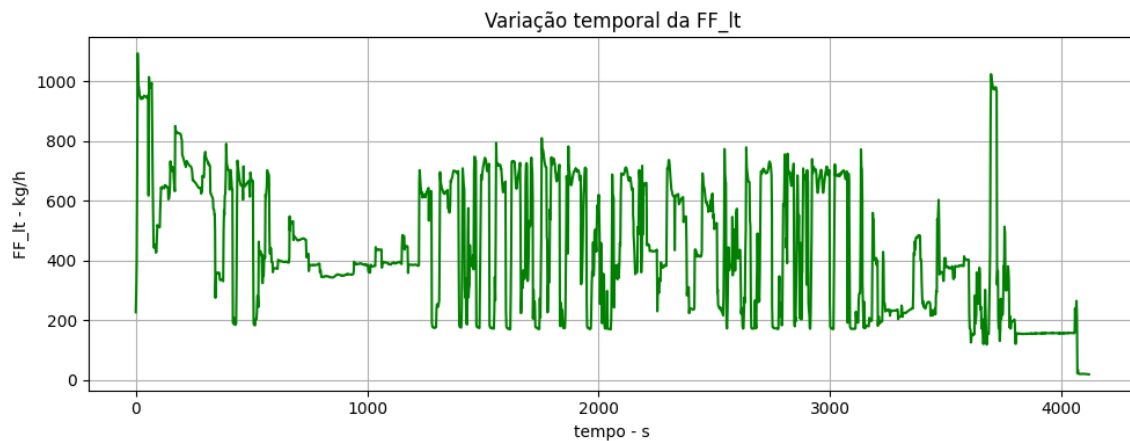


Figura 22: Consumo de combustível FF_{It}

De uma forma geral, os gráficos permitem identificar uma missão composta por diversas fases: uma fase inicial de decolagem e aceleração, seguida de uma subida até aproximadamente 15000 *ft*, depois uma fase prolongada de ensaios em altitude (fase central com manobras), e por fim uma descida e aproximação final. Sequência coerente com um ensaio em voo de uma aeronave sujeito a diferentes regimes de velocidade, altitude e solicitação dinâmica.

Durante grande parte do voo, os dois motores apresentam um comportamento semelhante nos parâmetros N2, EGT e FF. Logo o motor direito e o esquerdo operam de forma equilibrada e respondem de maneira semelhante às variações de potência.

A fase mais dinâmica do voo ocorre na zona intermédia do registo, aqui observamos grandes variações na aceleração vertical, na velocidade e nos parâmetros dos motores (N2, EGT, FF). Estes picos correspondem a manobras que se devem às variações rápidas de potência, o que mostra uma interação clara entre o comportamento dinâmico da aeronave e a resposta do sistema propulsivo.

Na parte final observa-se uma diferença entre os dois motores. O motor esquerdo apresenta uma redução acentuada dos parâmetros do sistema propulsivo (N2, EGT, FF), enquanto que o direito mantém os valores consideravelmente superiores. Esta assimetria corresponde a um corte de potência do motor esquerdo.

4.2 Pergunta 14 - Conversão da aceleração vertical para unidades *g*

De seguida, converteu-se a aceleração vertical, a_z , de m/s^2 para adimensional *g*. Esta conversão permite interpretar diretamente os fatores de carga aplicados à aeronave ao longo do voo.

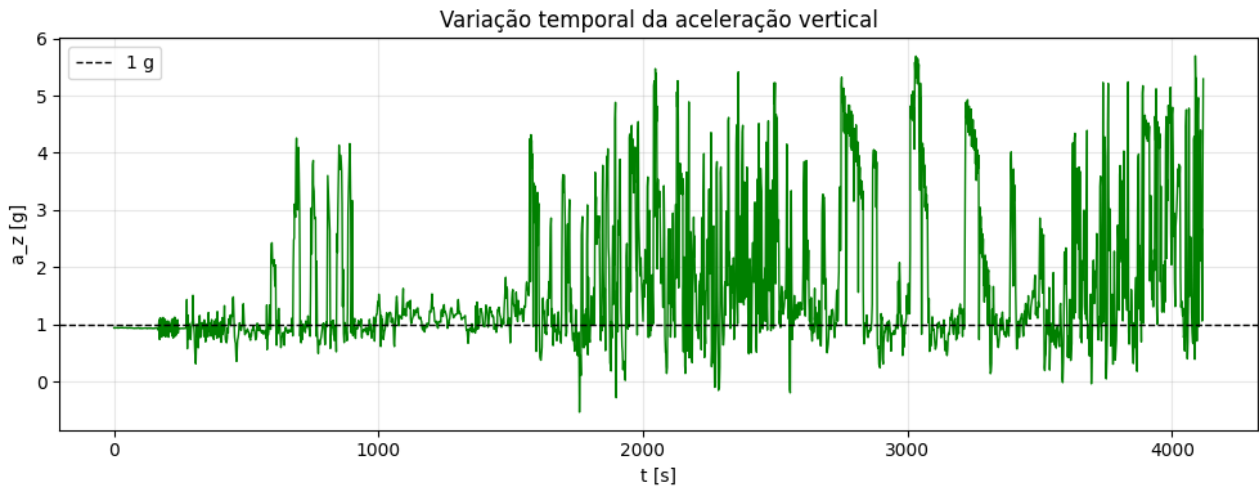


Figura 23: Variação temporal da aceleração vertical em g

4.3 Pergunta 15 - Extremos relativos da aceleração

De modo a obter uma análise mais detalhada da aceleração vertical, foi realizada uma identificação dos seus extremos relativos. Para tal, utilizou-se a biblioteca *SciPy*, que permite identificar os máximos e mínimos, posteriormente guardados no ficheiro `maximos_minimos.csv`.

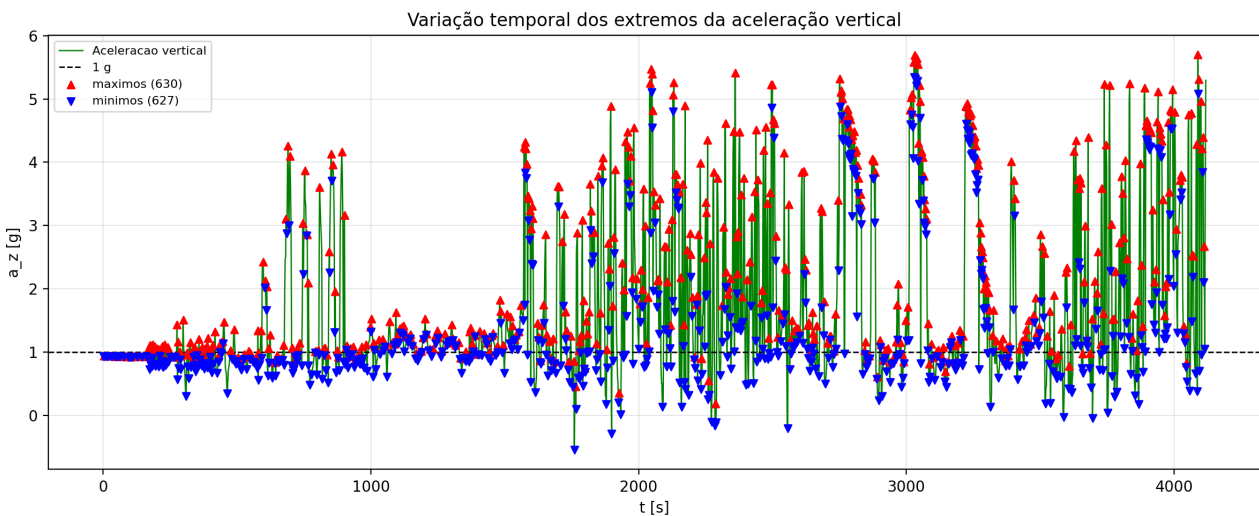


Figura 24: Variação temporal dos extremos da aceleração vertical em g

Podemos observar na Figura 24 os resultados obtidos, onde a linha verde representa a aceleração vertical em g, os pontos vermelhos representam os máximos e os pontos azuis representam os mínimos.

4.4 Pergunta 16 - Ciclos de aceleração vertical

O passo seguinte foi criar um algoritmo capaz de contar o número de ciclos de aceleração que ocorreram durante o ensaio. A ocorrência de um ciclo de aceleração $N1$ é iniciada quando o valor de az ultrapassar o valor estipulado para $N1$ ($az > N1$ se $N1 > 1g$ ou $az < N1$ se $N1 < 1g$), e dá-se como concluída quando az ultrapassar o valor estipulado para $N2$ ($az < N2$ se $N1 > 1g$ ou $az > N2$ se $N1 < 1g$). Os valores $N1$ e $N2$ são definidos pelo utilizador. O algoritmo encontra-se disponível no Anexo D.

4.5 Pergunta 17 - Pares de valores

A pedido do enunciado do trabalho experimental, aplicou-se o algoritmo a um conjunto de pares de valores. Os resultados podem ser consultados na Tabela 12 e visualizados na Figura 25.

N_1 (g)	N_2 (g)	Número de ciclos
2.5	2.2	124
4.0	3.7	59
5.0	4.7	17
6.0	5.7	0
7.0	6.7	0
0.0	0.3	7
-1.5	-1.2	0
-2.5	-2.2	0

Tabela 12: Número de ciclos para cada par de valores (N_1 , N_2)

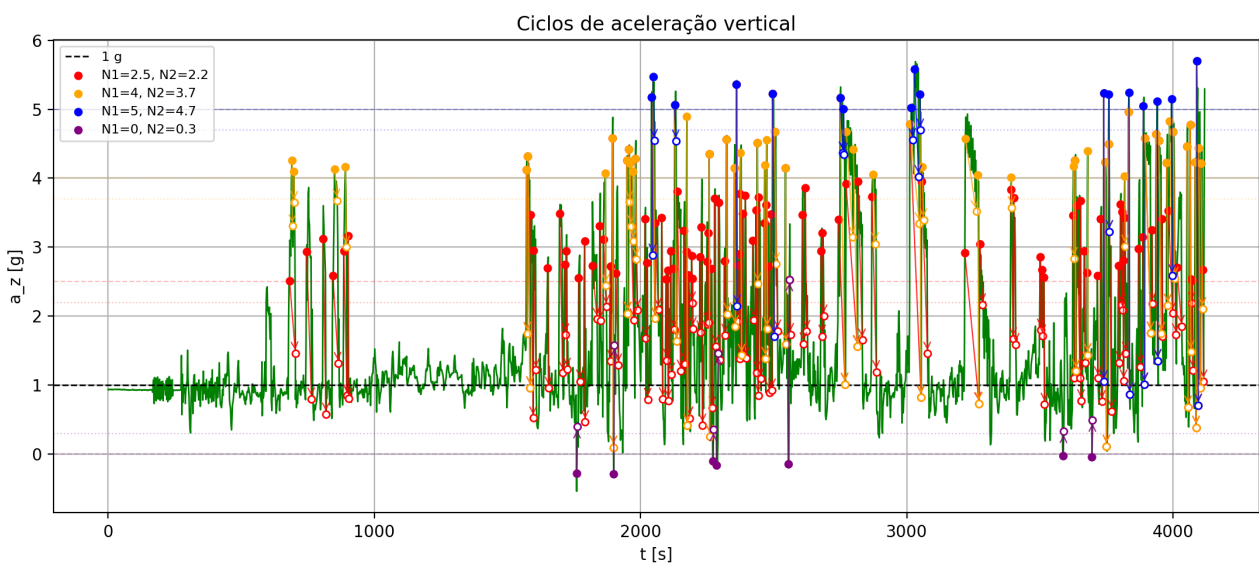


Figura 25: Ciclos de aceleração

4.6 Pergunta 18 - Ciclos de aceleração filtrados

Por fim, aplicamos o mesmo algoritmo de detecção dos ciclos, mas agora para o ficheiro criado na pergunta 15. Os resultados podem ser consultados na Tabela 13 e visualizados na Figura 26

N_1 (g)	N_2 (g)	Número de ciclos
2.5	2.2	124
4.0	3.7	59
5.0	4.7	17
6.0	5.7	0
7.0	6.7	0
0.0	0.3	7
-1.5	-1.2	0
-2.5	-2.2	0

Tabela 13: Número de ciclos para cada par de valores (N_1, N_2) extremos filtrados

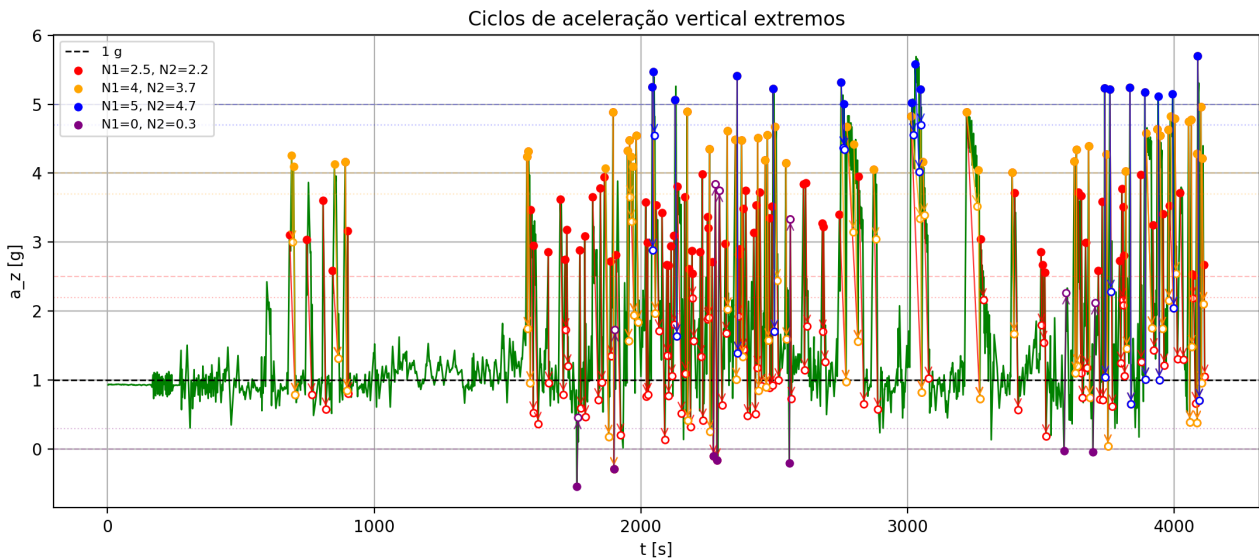


Figura 26: Ciclos de aceleração extremos filtrados.

4.7 Pergunta 19 - Comentários

Comparando os resultados das perguntas 17 e 18, verifica-se que o número de ciclos é igual para todos os pares de valores em ambos os casos. Isto acontece apesar de o ficheiro dos extremos, criado na pergunta 15, conter cerca de 1000 pontos a menos que o ficheiro original.

Isto acontece pois o algoritmo identifica um ciclo quando a aceleração ultrapassa o limite N_1 e posteriormente atravessa o limite N_2 . Como o processo preserva os extremos responsáveis por essas transições, a redução dos dados não altera o número de ciclos detetados. Contudo,

os instantes e valores exatos associados ao início e ao fim dos ciclos podem ser diferentes, uma vez que as amostras intermédias foram removidas.

O código desenvolvido pode ser consultado em Anexo D.

A Descrição dos campos do ficheiro EV_2026.A24

Tabela 14: Descrição dos campos utilizados na análise

Campo	Descrição
RX_TOM	Tempo da semana GPS (s)
RX_WEEK	Número da semana GPS
NSV_LOCK	Satélites observados
NSV_USED	Satélites utilizados na solução EGNOS
NS_HPL	Nível de proteção horizontal (m)
NS_VPL	Nível de proteção vertical (m)
NS_LAT, NS_LON, NS_ALT	Solução EGNOS
REF_LAT, REF_LON, REF_ALT	Trajetória de referência

B Código de Análise do Ficheiro EV_2026.A24

```

1 import pandas as pd
2 import numpy as np
3 from math import *
4 from pyproj import Transformer
5 from datetime import datetime, timedelta
6 import matplotlib.pyplot as plt
7 import matplotlib.dates as md
8
9 pd.set_option("display.float_format", "{:.10f}".format)
10
11
12 ### Escolha do método para conversão entre referenciais ###
13
14 def wgs84_latlonh_to_xyz(lon,lat,h):
15
16     a=6378137.0           #radius a of earth in meters cfr WGS84
17     b=6356752.3           #radius b of earth in meters cfr WGS84
18     e2=1-(b**2/a**2)      #squared excentricity
19     latr=lat/90*0.5*pi     #latitude in radians
20     lonr=lon/180*pi        #longituede in radians
21     Nphi=a/sqrt(1-e2*sin(latr)**2)
22     x=(Nphi+h)*cos(latr)*cos(lonr)
23     y=(Nphi+h)*cos(latr)*sin(lonr)
24     z=(b**2/a**2*Nphi+h)*sin(latr)
25     return([x,y,z])
26
27 print('Elipsoidal Formula: ', wgs84_latlonh_to_xyz(110. , -40. , 10))
28
29 #Or
30
31 lon, lat, hgt = 110., -40., 10 # Longitude, Latitude, Altitude
32 llh_to_xyz = Transformer.from_crs("EPSG:4979", "EPSG:4978", always_xy=True)
33
34 print('Using Transformer from pyproj: ', llh_to_xyz.transform(lon, lat, hgt)
35 )
36
37 ### Apresentar as coordenadas dos pontos em metros ###
38
39 data = {
40     "Ponto": ["Antena", "Solução A", "Solução B", "Solução C", "Solução D"],
41     "Longitude": ["-9.19497112", "-9.19496103", "-9.19497203", "-9.19497603", "-9.19497003"],
42     "Latitude": ["38.80572026", "38.80482050", "38.80572550", "38.80581050", "38.80562050"],
43     "Alt_ref": ["1282.192", "1284.221", "1281.321", "1280.221", "1282.221"]
44 }
45
46 df = pd.DataFrame(data)
47
48 df["Longitude"] = df["Longitude"].astype(float)
49 df["Latitude"] = df["Latitude"].astype(float)
50 df["Alt_ref"] = df["Alt_ref"].astype(float)

```

```

50
51 df[["X", "Y", "Z"]] = df.apply(
52     lambda row: pd.Series(
53         llh_to_xyz.transform(
54             row["Longitude"],
55             row["Latitude"],
56             row["Alt_ref"]
57         )
58     ),
59     axis=1
60 )
61
62 df.head() # mostra os 5 primeiros elementos
63
64 ### Função que implementa o algoritmo descrito no documento enviado ###
65
66 def get_error(xi, lon_ant, lat_ant, hgt_ant, lon, lat, hgt):
67
68     llh_to_xyz = Transformer.from_crs("EPSG:4979", "EPSG:4978", always_xy=
69     True)
70     b = llh_to_xyz.transform(lon_ant, lat_ant, hgt_ant) #Obter as
71     coordenadas XYZ da antena
72     c = llh_to_xyz.transform(lon, lat, hgt) #Obter as coordenadas XYZ da soluç
73     ão GNSS
74
75     b = np.array(b)
76     c = np.array(c)
77     # r = AB = B - A = B , como A é a origem
78     # s = AC = C - A = C
79
80     r = b
81     s = c
82
83     # calcular o erro horizontal (para efeito de teste)
84     d1 = np.linalg.norm(np.cross(s,r)) / np.linalg.norm(r)
85
86     # Isto seria a lógica para o d, mas temos que fazer considerando A' um
87     ponto mais próximo de B e C
88     # calcular os erros e retornar d_error e h_error
89
90     al = b - xi * r / np.linalg.norm(r)
91
92     rl = xi * r / np.linalg.norm(r)
93
94     sl = c - al #sl = vec(A'C)
95
96     #Erro de posição horizontal
97     d2 = np.linalg.norm(np.cross(sl,rl)) / np.linalg.norm(rl)
98
99     #Erro de posição vertical
100     h = sqrt(np.linalg.norm(sl)**2 - d2**2) - np.linalg.norm(rl)
101
102     return [d2 , h]
103
104 # Teste da função

```

```

101 print(get_error(10, -9.19497112, 38.80572026,
102         1282.192, -9.1949610300, 38.8048205000, 1284.2210000000 ))
103 ### Utilização da função acima no dataframe com as soluções propostas ###
104
105 df["X"] = df["X"].astype(float)
106 df["Y"] = df["Y"].astype(float)
107 df["Z"] = df["Z"].astype(float)
108
109 antena = [-9.19497112, 38.80572026, 1282.192] # lon, lat, hgt
110 xi = 10
111
112 df[["d_error", "h_error"]] = df.apply(
113     lambda row: pd.Series(
114         get_error(xi,
115             antena[0],
116             antena[1],
117             antena[2],
118             row["Longitude"],
119             row["Latitude"],
120             row["Alt_ref"]
121         )
122     ),
123     axis=1
124 )
125
126 df.head()
127
128 ### Definição da função Haversine e comparação com o método anterior ###
129
130 print(df[["d_error", "h_error"]])
131
132 # Função que será utilizada no exercício 4
133
134 #Fórmula de Haversine para aproximar o erro horizontal
135 def haversine (ref_lat, ref_lon, ns_lat, ns_lon):
136
137     lat1, lon1 = np.radians(ref_lat), np.radians(ref_lon)
138     lat2, lon2 = np.radians(ns_lat), np.radians(ns_lon)
139     R = 6371000
140
141     dlat = lat2 - lat1
142     dlon = lon2 - lon1
143
144     a = np.sin(dlat / 2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)
145     **2
146     c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
147     HPE1 = R * c
148
149     return HPE1
150
151 HPE = haversine (df["Latitude"], df["Longitude"], antena[1], antena[0]) #
152     antena[0] -> antena_longitude
153 VPE = df['Alt_ref'] - 1282.192

```

```

153 print("\nd_error_haversine\n" , HPE , "\n\nh_error_haversine\n",VPE)
154
155 ### Extração de variáveis do dataframe obtido do documento fornecido ###
156
157 df = pd.read_csv("EV_2026-A24.txt" , sep = ";")
158
159 # Extração de variáveis
160 RX_TOM = np.array(df["RX_TOM"])           # Receiver time of measurement
161 RX_WEEK = np.array(df["RX_WEEK"])         # GPS week number
162 NSV_LOCK = np.array(df["NSV_LOCK"])       # Number of satellites available
163 NSV_USED = np.array(df["NSV_USED"])       # Number of satellites used
164 NS_HPL = np.array(df["NS_HPL"] )         # Horizontal Protection Level
165 NS_VPL = np.array(df["NS_VPL"] )         # Vertical Protection Level
166 NS_LAT = np.array(df["NS_LAT"] )         # EGNOS latitude
167 NS_LON= np.array(df["NS_LON"] )         # EGNOS longitude
168 NS_ALT = np.array(df["NS_ALT"] )         # EGNOS altitude
169 REF_LAT = np.array(df["REF_LAT"])         # Reference latitude
170 REF_LON = np.array(df["REF_LON"])         # Reference longitude
171 REF_ALT = np.array(df["REF_ALT"])         # Reference altitude
172
173
174 #Erro Horizontal
175 HPE = haversine(REF_LAT, REF_LON, NS_LAT, NS_LON)
176
177 #Erro Vertical
178 VPE = NS_ALT - REF_ALT
179
180 ### Cálculo de timestamps e plots de HPE vs HPL e VPE vs VPL ###
181
182 # Calcular a data em timestamps
183 Epoch = datetime(1980 , 1 , 6)
184 week_start = Epoch + timedelta ( weeks = 1963)
185 timestamps = [ week_start + timedelta ( seconds = t ) for t in df["RX_TOM"]]
186
187 def time_format():
188     ax=plt.gca()
189     xfmt = md.DateFormatter('%Y-%m-%d %H:%M:%S')
190     ax.xaxis.set_major_formatter(xfmt)
191     plt.gcf().autofmt_xdate()
192     return
193
194
195 # Erro Plano Horizontal
196 plt.figure(figsize=(12,6))
197 plt.plot(timestamps, HPE, label = 'HPE')
198 plt.plot(timestamps, NS_HPL, color = 'r', label = 'HPL')
199 plt.grid()
200 plt.legend()
201 plt.ylabel("Horizontal Error [m]")
202 plt.xlabel("Time")
203 plt.title('HPE and HPL vs Time')
204 time_format()
205
206
207 # Erro Plano Vertical

```



```

208 VPE = [abs(x) for x in VPE] # Para poder comparar com o VPL
209
210 plt.figure(figsize=(12,6))
211 plt.plot(timestamps, VPE, label = 'VPE')
212 plt.plot(timestamps, NS_VPL, color = 'r', label = 'VPL')
213 plt.grid()
214 plt.legend()
215 plt.ylabel("Vertical Error [m]")
216 plt.xlabel("Time")
217 plt.title('VPE and VPL vs Time')
218 time_format()
219
220 ### Plot de satélites observados e utilizados para fornecer a solução ###
221
222 plt.figure(figsize=(10,5))
223 plt.plot(timestamps, NSV_USED, color = 'b', label = 'Used Satelites')
224 plt.plot(timestamps, NSV_LOCK, color = 'orange', label = 'Available
    Satelites')
225 plt.grid()
226 plt.legend()
227 plt.ylabel("Satelites Number")
228 plt.xlabel("Time")
229 plt.title('Observed & Used Satelites vs Time')
230 time_format()
231
232 ### Cálculo dos percentis dos erros de posição horizontal (HPE) e vertical (
    VPE) ###
233
234 p95_HPE = np.percentile(HPE, 95)
235 p95_VPE = np.percentile(VPE, 95)
236
237 print(f"HPE Percentil 95\%: {p95_HPE}")
238 print(f"VPE Percentil 95\%: {p95_VPE}")
239
240 ### Cálculo dos percentis dos níveis de proteção (NS_HPL e NS_VPL) ###
241
242 p99_NS_HPL = np.percentile(NS_HPL, 99)
243 p99_NS_VPL = np.percentile(NS_VPL, 99)
244
245 print(f"NS_HPL Percentil 99\%: {p99_NS_HPL}")
246 print(f"NS_VPL Percentil 99\%: {p99_NS_VPL}")
247
248 ### Plot para observar a disponibilidade dos modos de operação ###
249
250 HAL_APV_I = 40
251 HAL_APV_II = 40
252 HAL_CAT_I = 40
253
254 VAL_APV_I = 50
255 VAL_APV_II = 20
256 VAL_CAT_I = 12
257
258 # Disponibilidade dos níveis horizontais
259 plt.figure(figsize=(10,5))
260 plt.plot(timestamps, NS_HPL, color = 'b', label = 'Horizontal Protection

```

```

    Level')
261 plt.axhline(HAL_APV_I, color = 'r', linestyle = "--", label = 'Horizontal
    Alert Level')
262 plt.grid()
263 plt.legend()
264 plt.ylabel("HPL")
265 plt.xlabel("Time")
266 plt.title('Horizontal Alert and Protection Levels')
267 time_format()
268
269 # Disponibilidade dos níveis verticais
270 plt.figure(figsize=(10,5))
271 plt.plot(timestamps, NS_VPL, color = 'b', label = 'Vertical Protection Level
    ')
272 plt.axhline(VAL_APV_I, color = 'yellow', linestyle = "--", label = 'Vertical
    APV_I Alert Level')
273 plt.axhline(VAL_APV_II, color = 'orange', linestyle = "--", label = '
    Vertical APV_II Alert Level')
274 plt.axhline(VAL_CAT_I, color = 'red', linestyle = "--", label = 'Vertical
    CAT_I Alert Level')
275 plt.grid()
276 plt.legend()
277 plt.ylabel("VPL")
278 plt.xlabel("Time")
279 plt.title('Vertical Alert and Protection Levels')
280 time_format()
281
282 ### Cálculo da disponibilidade dos modos de operação ###
283
284 #Função get_availability
285
286 def get_availability(RX_TOMf, HALf, VALf, HPLf, VPLf):
287     t = len(RX_TOMf)
288     count = 0
289
290     for i in range(len(RX_TOMf)):
291         if ((HPLf[i] < HALf) and (VPLf[i] < VALf)):
292             count = count + 1
293
294     return (count/t * 100)
295
296 APV_I_availability = get_availability(RX_TOM, HAL_APV_I, VAL_APV_I, NS_HPL,
    NS_VPL)
297 APV_II_availability = get_availability(RX_TOM, HAL_APV_II, VAL_APV_II,
    NS_HPL, NS_VPL)
298 CAT_I_availability = get_availability(RX_TOM, HAL_CAT_I, VAL_CAT_I, NS_HPL,
    NS_VPL)
299
300
301 print('APV_I_availability:', (APV_I_availability), '%')
302 print('APV_II_availability:', (APV_II_availability), '%')
303 print('CAT_I_availability:', (CAT_I_availability), '%')
304
305 ### Definição e utilização da função que contabiliza os intervalos de
    continuidade ###

```

```

306
307 def get_continuity(RX_TOMf, HALf, VALf, HPLf, VPLf):
308     vec = []
309     for i in range(len(RX_TOMf)):
310         if ((HPLf[i] >= HALf) or (VPLf[i] >= VALf)):
311             vec.append(0) # Indisponível
312         else:
313             vec.append(1) # Disponível
314
315     count = 0
316     if len(vec) > 0 and vec[0] == 1:
317         count = 1
318
319     # Verifica transições de 0 para 1 (o sistema voltou a ficar disponível)
320     for j in range(1, len(vec)):
321         if vec[j] == 1 and vec[j-1] == 0:
322             count += 1
323
324     return count
325
326 APV_I_continuity = get_continuity(RX_TOM, HAL_APV_I, VAL_APV_I, NS_HPL,
327                                   NS_VPL)
328 APV_II_continuity = get_continuity(RX_TOM, HAL_APV_II, VAL_APV_II, NS_HPL,
329                                    NS_VPL)
330 CAT_I_continuity = get_continuity(RX_TOM, HAL_CAT_I, VAL_CAT_I, NS_HPL,
331                                   NS_VPL)
332
333 print('APV_I_continuity:', (APV_I_continuity))
334 print('APV_II_continuity:', (APV_II_continuity))
335 print('CAT_I_continuity:', (CAT_I_continuity))
336
337 ### Definição e implementação de função que identifica eventos de
338     integridade ###
339
340 def check_integrity(RX_TOM, RX_WEEK, HPEf, VPEf, HPLf, VPLf):
341
342     epoch = datetime(1980, 1, 6) # Epoch GPS (06/01/1980)
343     def gps_to_datetime(week, seconds):
344         return [epoch + timedelta(weeks=int(w), seconds=int(s)) for w, s in
345                 zip(week, seconds)]
346
347     timestamps = np.array(gps_to_datetime(RX_WEEK, RX_TOM))
348
349     #Lógica de integridade vetorizada
350     violation_H = HPEf > HPLf
351     violation_V = VPEf > VPLf
352
353     status_code = np.zeros(len(HPEf), dtype=int)
354     status_code[violation_H & ~violation_V] = 1
355     status_code[~violation_H & violation_V] = 2
356     status_code[violation_H & violation_V] = 3
357
358     indices_erro = np.where(status_code > 0)[0]
359
360     print(f"Total de eventos de integridade encontrados: {len(indices_erro)}")

```

```
    ")
356
357     print(HPEf[indices_erro])
358
359     for i in indices_erro:
360         motivo = {1: "Violação HPL", 2: "Violação VPL", 3: "Violação HPL e
Violação VPL"}[status_code[i]]
361         print(f>Data: {timestamps[i]} | Motivo: {motivo} | HPE: {HPEf[i]} |
VPE: {VPEf[i]}")
362
363     return timestamps, status_code
364
365 status = check_integrity(RX_TOM, RX_WEEK, HPE, VPE, NS_HPL, NS_VPL)
366
367 ### Fim de Código de análise do documento EV2026.A24 ###
```

C Código de Análise do Ficheiro EV_2026.B24

```

1 import numpy as np
2 from scipy.fftpack import fft, fftfreq, rfft
3 from scipy.signal import find_peaks
4 import matplotlib.pyplot as plt
5 import pandas as pd
6
7 IMAGE_SIZE = (10, 3.5)
8 SHOW_IMAGES = False
9
10
11 def main():
12     # read csv to dataframe
13     df = pd.read_csv("EV_2026.B24", sep=";")
14
15     t = df.iloc[:,0] # in seconds
16
17     sinais = list(df.columns.values)[1:]
18     for i in range(len(sinais)):
19         # in m /s^2
20         acel_i = df.iloc[:, i + 1] # skip 0, cause is time
21
22         # 9. variacao temporal das grandezas
23         plot_sinal_graph(t, acel_i, sinais[i])
24
25         # 10. determinar e apresentar o espetro unilateral de amplitude
26         (f, Y) = FFT(t, acel_i.values)
27         plot_single_sided_spectrum(f, Y, sinais[i])
28
29
30 def FFT(t, y):
31     N = len(t) # data length
32     T = t[1] - t[0] # from data
33     # espetro unilateral de magnitude, de 0 ate freq de Nyquist
34     # 0 resto corresponde freq negativa da transformada de fourier, real
35     # signal, is a mirror
36     n_uni = N//2
37     f = fftfreq(N, T)[:n_uni]
38
39     # FFT to get: single-sided magnitude spectrum
40     Y = fft(y)
41
42     # To get single-sided magnitude spectrum, we need to do:
43     # The two-sided amplitude spectrum, where the spectrum in the positive
44     # frequencies is the complex conjugate of the spectrum in the negative
45     # frequencies, has half the peak amplitudes of the time-domain signal.
46     # Also, we need to rescale, dividing by N
47     Y = 2 / N * np.abs(Y[:n_uni])
48     # DC (0 Hz) is unique in the FT
49     Y[0] = Y[0] / 2
50     # Nyquist frequency is unique for even N, so we shall not multiply by 2,
51     # so we revert the previous multiplication
52     if N % 2 == 0:

```

```

49     Y[-1] = Y[-1] / 2
50     return (f, Y)
51
52
53 def get_freq_peaks(f, y, signal_name, height_threshold=0.05):
54     peaks_index, properties = find_peaks(np.abs(y), height=height_threshold)
55     print('Peaks of signal ' + signal_name)
56     print("Frequency: \t Magnitude:")
57     [print("%4.4f \t %3.4f" %(f[peaks_index[i]], properties['peak_heights']
58     '[i])) for i in range(len(peaks_index))]
59     return peaks_index, properties
60
61 def plot_sinal_graph(t, signal, signal_name):
62     # Visualization
63     plt.figure(figsize=IMAGE_SIZE)
64
65     # Original signal
66     plt.plot(t, signal.values, linewidth=0.5)
67     plt.title("Variação temporal do sinal " + signal_name)
68     plt.xlabel("Tempo [s]")
69     plt.ylabel("Aceleracao [m/s^2]")
70     plt.grid()
71
72     plt.tight_layout()
73     plt.savefig("results/sinal-"+signal_name+".png")
74     if SHOW_IMAGES:
75         plt.show()
76
77
78 def plot_single_sided_spectrum(f, ftt_signal, signal_name):
79
80     # get peaks
81     peaks_index, properties = get_freq_peaks(f, ftt_signal, signal_name)
82
83     # Visualization
84     plt.figure(figsize=IMAGE_SIZE)
85
86     # Spectral
87     plt.plot(f, ftt_signal, '-', f[peaks_index], properties['peak_heights'], 'x')
88     plt.title("Espectro unilateral de amplitude do sinal " + signal_name)
89     plt.xlabel("Freq [Hz]")
90     plt.ylabel("Magnitude |X(freq)|")
91     plt.grid()
92
93     plt.tight_layout()
94     plt.savefig("results/espectro-"+signal_name+".png")
95     if SHOW_IMAGES:
96         plt.show()
97
98
99 if __name__ == "__main__":
100     main()

```

D Código de Análise do Ficheiro EV_2026.C24

```

1
2 # 13
3 from pathlib import Path
4
5 import matplotlib.pyplot as plt
6 import pandas as pd
7
8
9 DATA_FILE = Path(__file__).with_name("EV_2026.C24")
10
11
12 def make_plot(t, y, title, ylabel):
13     fig, ax = plt.subplots(figsize=(10, 4))
14     ax.plot(t, y, color="g")
15     ax.set_title(title)
16     ax.set_xlabel("tempo - s")
17     ax.set_ylabel(ylabel)
18     ax.grid(True)
19     fig.tight_layout()
20
21
22 df = pd.read_csv(DATA_FILE, sep=";")
23
24 t = df["t"]
25
26 plots = [
27     ("EAS", "EAS - kn", "Variação temporal da EAS"),
28     ("QNE", "QNE - ft", "Variação temporal da QNE"),
29     ("a_z", "a_z - m/s^2", "Variação temporal da a_z"),
30     ("N2_rh", "N2_rh - %", "Variação temporal da N2_rh"),
31     ("EGT_rh", "EGT_rh - K", "Variação temporal da EGT_rh"),
32     ("FF_rh", "FF_rh - kg/h", "Variação temporal da FF_rh"),
33     ("N2_lt", "N2_lt - %", "Variação temporal da N2_lt"),
34     ("EGT_lt", "EGT_lt - K", "Variação temporal da EGT_lt"),
35     ("FF_lt", "FF_lt - kg/h", "Variação temporal da FF_lt"),
36 ]
37
38 for column, ylabel, title in plots:
39     make_plot(t, df[column], title, ylabel)
40
41 plt.show()
42
43 # 14
44 from pathlib import Path
45
46 import matplotlib.pyplot as plt
47 import pandas as pd
48
49
50 DATA_FILE = Path(__file__).with_name("EV_2026.C24")
51 CSV_OUTPUT_FILE = DATA_FILE.with_name("EV_2026.C24_g_values")
52 GRAPH_OUTPUT_FILE = DATA_FILE.with_name("EV_2026.C24_g_values.png")

```

```

53
54 G_0 = 9.80665 # m/s^2
55
56 df = pd.read_csv(DATA_FILE, sep=";")
57
58 required_columns = {"t", "a_z"}
59 missing_columns = required_columns - set(df.columns)
60
61 if missing_columns:
62     raise ValueError(f"Missing columns in {DATA_FILE.name}: {sorted(
63         missing_columns)}")
64
65 for column in required_columns:
66     df[column] = pd.to_numeric(df[column], errors="coerce")
67
68 df = df.dropna(subset=required_columns).sort_values("t").reset_index(drop=
69     True)
70 df["a_z_g"] = df["a_z"] / G_0
71
72 g_values = df[["t", "a_z", "a_z_g"]]
73 g_values.to_csv(CSV_OUTPUT_FILE, sep=";", index=False)
74
75 fig, ax = plt.subplots(figsize=(10, 4))
76 ax.plot(g_values["t"], g_values["a_z_g"], color="green", linewidth=1)
77 ax.axhline(1.0, color="black", linestyle="--", linewidth=1, label="1 g")
78 ax.set_title("Variação temporal da aceleração vertical")
79 ax.set_xlabel("t [s]")
80 ax.set_ylabel("a_z [g]")
81 ax.grid(True, alpha=0.3)
82 ax.legend()
83
84 fig.tight_layout()
85 fig.savefig(GRAPH_OUTPUT_FILE, dpi=300)
86
87 print(g_values.to_string(index=False))
88 print(f"\nCalculated {len(g_values)} g values.")
89 print(f"Saved results to {CSV_OUTPUT_FILE.name}.")
90 print(f"Saved graph to {GRAPH_OUTPUT_FILE.name}.")
91
92 plt.show()
93
94 # 15
95
96 from pathlib import Path
97
98 import matplotlib.pyplot as plt
99 import pandas as pd
100 from scipy.signal import find_peaks
101
102 DATA_FILE = Path(__file__).with_name("EV_2026.C24_g_values")
103 OUTPUT_FILE = Path(__file__).with_name("maximos_minimos")
104 PLOT_FILE = Path(__file__).with_name("maximos_minimos.png")
105
106 df = pd.read_csv(DATA_FILE, sep=";")

```



```

106 colunas_necessarias = {"t", "a_z", "a_z_g"}
107 colunas_em_falta = colunas_necessarias - set(df.columns)
108
109 if colunas_em_falta:
110     raise ValueError(
111         f"Faltam as seguintes colunas no ficheiro: {sorted(colunas_em_falta)}"
112     )
113
114 for coluna in colunas_necessarias:
115     df[coluna] = pd.to_numeric(df[coluna], errors="coerce")
116
117 df = df.dropna(subset=colunas_necessarias).reset_index(drop=True)
118
119 valores_a_z_g = df["a_z_g"].to_numpy()
120
121 indices_maximos, _ = find_peaks(valores_a_z_g, plateau_size=(None, 1))
122 indices_minimos, _ = find_peaks(-valores_a_z_g, plateau_size=(None, 1))
123
124 df["tipo"] = ""
125 df.loc[indices_maximos, "tipo"] = "max"
126 df.loc[indices_minimos, "tipo"] = "min"
127
128 maximos_minimos = df.loc[df["tipo"] != "", ["tipo", "t", "a_z", "a_z_g"]]
129
130 maximos_minimos.to_csv(OUTPUT_FILE, sep=";", index=False)
131
132 dados_maximos = df.loc[indices_maximos]
133 dados_minimos = df.loc[indices_minimos]
134
135 fig, ax = plt.subplots(figsize=(12, 5))
136 ax.plot(
137     df["t"],
138     df["a_z_g"],
139     color="green",
140     linewidth=0.9,
141     label="Aceleracao vertical",
142 )
143 ax.axhline(1.0, color="black", linestyle="--", linewidth=1, label="1 g")
144 ax.scatter(
145     dados_maximos["t"],
146     dados_maximos["a_z_g"],
147     color="red",
148     marker="^",
149     s=24,
150     zorder=3,
151     label=f"maximos ({len(dados_maximos)})",
152 )
153 ax.scatter(
154     dados_minimos["t"],
155     dados_minimos["a_z_g"],
156     color="blue",
157     marker="v",
158     s=24,
159     zorder=3,

```

```

160     label=f"minimos ({len(dados_minimos)})",
161 )
162
163 ax.set_title("Variação temporal dos extremos da aceleração vertical")
164 ax.set_xlabel("t [s]")
165 ax.set_ylabel("a_z [g]")
166 ax.grid(True, alpha=0.3)
167 ax.legend(fontsize=8)
168
169 fig.tight_layout()
170 fig.savefig(PLOT_FILE, dpi=200)
171
172 numero_maximos = (maximos_minimos["tipo"] == "max").sum()
173 numero_minimos = (maximos_minimos["tipo"] == "min").sum()
174
175 print(f"maximos da trajetoria encontrados: {numero_maximos}")
176 print(f"minimos da trajetoria encontrados: {numero_minimos}")
177 print(f"Total de eventos encontrados: {len(maximos_minimos)}")
178 print(f"Resultados guardados em: {OUTPUT_FILE.name}")
179 print(f"Grafico guardado em: {PLOT_FILE.name}")
180
181 plt.show()
182
183 # 16-17
184
185 from pathlib import Path
186
187 import matplotlib.pyplot as plt
188 import pandas as pd
189 from scipy.signal import find_peaks
190
191 DATA_FILE = Path(__file__).with_name("EV_2026.C24_g_values")
192 OUTPUT_FILE = Path(__file__).with_name("maximos_minimos")
193 PLOT_FILE = Path(__file__).with_name("maximos_minimos.png")
194
195 df = pd.read_csv(DATA_FILE, sep=";")
196
197 colunas_necessarias = {"t", "a_z", "a_z_g"}
198 colunas_em_falta = colunas_necessarias - set(df.columns)
199
200 if colunas_em_falta:
201     raise ValueError(
202         f"Faltam as seguintes colunas no ficheiro: {sorted(colunas_em_falta)}"
203     )
204
205 for coluna in colunas_necessarias:
206     df[coluna] = pd.to_numeric(df[coluna], errors="coerce")
207
208 df = df.dropna(subset=colunas_necessarias).reset_index(drop=True)
209
210 valores_a_z_g = df["a_z_g"].to_numpy()
211
212 indices_maximos, _ = find_peaks(valores_a_z_g, plateau_size=(None, 1))
213 indices_minimos, _ = find_peaks(-valores_a_z_g, plateau_size=(None, 1))

```

```

214
215 df["tipo"] = ""
216 df.loc[indices_maximos, "tipo"] = "max"
217 df.loc[indices_minimos, "tipo"] = "min"
218
219 maximos_minimos = df.loc[df["tipo"] != "", ["tipo", "t", "a_z", "a_z_g"]]
220
221 maximos_minimos.to_csv(OUTPUT_FILE, sep=";", index=False)
222
223 dados_maximos = df.loc[indices_maximos]
224 dados_minimos = df.loc[indices_minimos]
225
226 fig, ax = plt.subplots(figsize=(12, 5))
227 ax.plot(
228     df["t"],
229     df["a_z_g"],
230     color="green",
231     linewidth=0.9,
232     label="Aceleracao vertical",
233 )
234 ax.axhline(1.0, color="black", linestyle="--", linewidth=1, label="1 g")
235 ax.scatter(
236     dados_maximos["t"],
237     dados_maximos["a_z_g"],
238     color="red",
239     marker="^",
240     s=24,
241     zorder=3,
242     label=f"maximos ({len(dados_maximos)})",
243 )
244 ax.scatter(
245     dados_minimos["t"],
246     dados_minimos["a_z_g"],
247     color="blue",
248     marker="v",
249     s=24,
250     zorder=3,
251     label=f"minimos ({len(dados_minimos)})",
252 )
253
254 ax.set_title("Variação temporal dos extremos da aceleração vertical")
255 ax.set_xlabel("t [s]")
256 ax.set_ylabel("a_z [g]")
257 ax.grid(True, alpha=0.3)
258 ax.legend(fontsize=8)
259
260 fig.tight_layout()
261 fig.savefig(PLOT_FILE, dpi=200)
262
263 numero_maximos = (maximos_minimos["tipo"] == "max").sum()
264 numero_minimos = (maximos_minimos["tipo"] == "min").sum()
265
266 print(f"maximos da trajetoria encontrados: {numero_maximos}")
267 print(f"minimos da trajetoria encontrados: {numero_minimos}")
268 print(f"Total de eventos encontrados: {len(maximos_minimos)}")

```

```
269 print(f"Resultados guardados em: {OUTPUT_FILE.name}")
270 print(f"Grafico guardado em: {PLOT_FILE.name}")
271
272 plt.show()
273
274 # 16-18
275
276 from pathlib import Path
277
278 import matplotlib.pyplot as plt
279 import pandas as pd
280
281
282 DATA_FILE = Path(__file__).with_name("maximos_minimos")
283 OUTPUT_FILE = Path(__file__).with_name("ciclos_a_z_g_filtrado")
284 PLOT_FILE = Path(__file__).with_name("ciclos_a_z_g_filtrado.png")
285
286 N_PAIRS = {
287     (2.5, 2.2): "red",
288     (4, 3.7): "orange",
289     (5.0, 4.7): "blue",
290     (6.0, 5.7): "brown",
291     (7.0, 6.7): "grey",
292     (0.0, 0.3): "purple",
293     (-1.5, -1.2): "cyan",
294     (-2.5, -2.2): "pink",
295 }
296
297
298 T_MIN = 0
299 T_MAX = 4400
300
301
302 def contar_ciclos(dados_df, n1, n2):
303     if n1 > 1.0 and n2 >= n1:
304         raise ValueError("Para N1 > 1 g, N2 deve ser inferior a N1.")
305     if n1 < 1.0 and n2 <= n1:
306         raise ValueError("Para N1 < 1 g, N2 deve ser superior a N1.")
307     if n1 == 1.0:
308         raise ValueError("0 algoritmo precisa de N1 diferente de 1 g.")
309
310     em_ciclo = False
311     inicio = None
312     ciclos = []
313
314     for linha in dados_df.itertuples(index=False):
315         t = linha.t
316         a_z_g = linha.a_z_g
317
318         if n1 > 1.0:
319             inicia = a_z_g > n1
320             fecha = a_z_g < n2
321         else:
322             inicia = a_z_g < n1
323             fecha = a_z_g > n2
```

```

324         if not em_ciclo and inicia:
325             em_ciclo = True
326             inicio = (t, a_z_g)
327             continue
328
329
330         if em_ciclo and fecha:
331             ciclos.append(
332                 {
333                     "N1": n1,
334                     "N2": n2,
335                     "t_inicio_amostra": inicio[0],
336                     "a_z_g_inicio_amostra": inicio[1],
337                     "t_fim_amostra": t,
338                     "a_z_g_fim_amostra": a_z_g,
339                 }
340             )
341             em_ciclo = False
342             inicio = None
343
344         return ciclos
345
346
347 df = pd.read_csv(DATA_FILE, sep=";")
348
349 colunas_necessarias = {"t", "a_z_g"}
350 colunas_em_falta = colunas_necessarias - set(df.columns)
351
352 if colunas_em_falta:
353     raise ValueError(f"Faltam colunas no ficheiro: {sorted(colunas_em_falta)}")
354
355 for coluna in colunas_necessarias:
356     df[coluna] = pd.to_numeric(df[coluna], errors="coerce")
357
358 df = df.dropna(subset=colunas_necessarias).sort_values("t").reset_index(drop=True)
359
360 todos_os_ciclos = []
361 ciclos_por_par = {}
362
363 for n1, n2 in N_PAIRS:
364     ciclos = contar_ciclos(df, n1, n2)
365     ciclos_por_par[(n1, n2)] = ciclos
366     todos_os_ciclos.extend(ciclos)
367     print(f"N1 = {n1:4.1f} g, N2 = {n2:4.1f} g: {len(ciclos)} ciclo(s)")
368
369 pd.DataFrame(todos_os_ciclos).to_csv(OUTPUT_FILE, sep=";", index=False)
370 print(f"\nResultados guardados em: {OUTPUT_FILE.name}")
371
372 df_plot = df.copy()
373
374 if T_MIN is not None:
375     df_plot = df_plot[df_plot["t"] >= T_MIN]
376 if T_MAX is not None:

```

```

377     df_plot = df_plot[df_plot["t"] <= T_MAX]
378
379 if df_plot.empty:
380     raise ValueError("O intervalo definido por T_MIN/T_MAX nao contem dados.
381 ")
382
381 t_min_plot = df_plot["t"].min()
382 t_max_plot = df_plot["t"].max()
383
384
385 fig, ax = plt.subplots(figsize=(11, 5))
386 ax.plot(df_plot["t"], df_plot["a_z_g"], color="g", linewidth=1)
387 ax.axhline(1.0, color="black", linestyle="--", linewidth=1, label="1 g")
388
389 for (n1, n2), ciclos in ciclos_por_par.items():
390     cor = N_PAIRS[(n1, n2)]
391     ciclos_visiveis = [
392         ciclo
393         for ciclo in ciclos
394         if t_min_plot <= ciclo["t_inicio_amostra"] <= t_max_plot
395         and t_min_plot <= ciclo["t_fim_amostra"] <= t_max_plot
396     ]
397
398     print(
399         f"N1 = {n1:4.1f} g, N2 = {n2:4.1f} g: "
400         f"{len(ciclos_visiveis)} ciclo(s)"
401     )
402
403     if not ciclos_visiveis:
404         continue
405
406     ax.axhline(n1, color=cor, linestyle="--", linewidth=0.8, alpha=0.25)
407     ax.axhline(n2, color=cor, linestyle=":", linewidth=0.8, alpha=0.25)
408
409     inicios_t = [ciclo["t_inicio_amostra"] for ciclo in ciclos_visiveis]
410     inicios_a_z = [ciclo["a_z_g_inicio_amostra"] for ciclo in
411 ciclos_visiveis]
412     fins_t = [ciclo["t_fim_amostra"] for ciclo in ciclos_visiveis]
413     fins_a_z = [ciclo["a_z_g_fim_amostra"] for ciclo in ciclos_visiveis]
414
415     ax.scatter(
416         inicios_t,
417         inicios_a_z,
418         color=cor,
419         s=18,
420         zorder=3,
421         label=f"N1={n1:g}, N2={n2:g}",
422     )
423     ax.scatter(fins_t, fins_a_z, facecolors="white", edgecolors=cor, s=18,
424 zorder=3)
425
426     for ciclo in ciclos_visiveis:
427         ax.annotate(
428             "",
429             xy=(ciclo["t_fim_amostra"], ciclo["a_z_g_fim_amostra"]),
430             xytext=(ciclo["t_inicio_amostra"], ciclo["a_z_g_inicio_amostra"]

```

```
    ]),  
429         arrowprops={"arrowstyle": "->", "color": cor, "lw": 0.8, "alpha"  
430         : 0.8},  
431     )  
432 ax.set_title("Ciclos de aceleração vertical extremos")  
433 ax.set_xlabel("t [s]")  
434 ax.set_ylabel("a_z [g]")  
435 ax.grid(True)  
436 ax.legend(fontsize=8)  
437  
438 fig.tight_layout()  
439 fig.savefig(PLOT_FILE, dpi=200)  
440 print(f"Grafico guardado em: {PLOT_FILE.name}")  
441 plt.show()
```

Referências

- ACMAA, Instituto Superior Técnico – DEM – (2025a). *Ensaio em Voo – 2025-2026*. Rel. téc. Grupo 24 – Trabalho experimental envolvendo processamento e análise de dados. Instituto Superior Técnico.
- ACMAA, Instituto Superior Técnico – DEM – (2025b). *EV – Avaliação da exatidão de um sistema GNSS*. Rel. téc. Documento de trabalho experimental 2025-2026. Instituto Superior Técnico.
- EPSG.io (2026a). *EPSG:4978 - WGS 84 (Geocêntrico e Cartesiano)*. <https://epsg.io/4978>. Acedido em: 8 de junho de 2026.
- EPSG.io (2026b). *EPSG:4979 - WGS 84 (Geocêntrico e Elipsoidal)*. <https://epsg.io/4979>. Acedido em: 8 de junho de 2026.
- International Civil Aviation Organization (2023). *Annex 10 to the Convention on International Civil Aviation: Aeronautical Telecommunications – Volume I: Radio Navigation Aids*. Montreal, Canada.
- Quora Contributors (2020). *How does the WGS-84 coordinate system work in layman's terms?* <https://www.quora.com/How-does-the-WGS-84-coordinate-system-work-in-layman-s-terms>. Acedido em: 8 de junho de 2026.
- Stack Overflow Contributors (2010). *Converting from longitude/latitude to Cartesian coordinates (Using Transformer from pyproj)*. <https://stackoverflow.com/questions/1185408/converting-from-longitude-latitude-to-cartesian-coordinates>. Acedido em: 8 de junho de 2026.